

컴퓨터 비전 프로젝트

EfficientDet 실습

코드 정리와 코드 리뷰

사전학습 모델 추론 → 영상 분석 → 차량 데이터 학습·평가 → 실습 코드 리뷰

작성자	hwang-ye-song
원본 자료	4. EfficientDet 실습 [프로젝트] (.ipynb 코드 · .pdf 강의 자료)
작성일	2026-10-04
실행 환경	원본은 Google Colab용 코드다. 내 컴퓨터(Windows 11 · Python 3.11 · PyTorch 2.11 CUDA 12.8 · RTX 5060 Laptop)에서 실행했고, 이를 위해 바꾼 줄은 코드 안에 💻 코랩용(원본) / 🖥️ 노트북용으로 나란히 표시했다

목차

1. Part 1. 사전학습 모델로 이미지 추론 (🔍 코드 리뷰 1-1 ~ 1-6)
2. Part 2. 영상 분석 (🔍 코드 리뷰 2-1 ~ 2-7)
3. Part 3. Custom dataset 학습 (차량) (🔍 코드 리뷰 3-1 ~ 3-12, 리뷰 근거 확인)
4. 코드 리뷰 종합 · 목표 기준 평가 · 진행하면서 막혔던 부분과 해결 · 개인 회고

EfficientDet 실습 [프로젝트] — 실습 코드 정리와 코드 리뷰

항목	내용
목표	① EfficientDet의 추론 과정(전처리 → 모델 → 후처리)을 코드로 이해한다 ② 사전학습 모델로 이미지·영상을 분석한다 ③ 차량 데이터셋(COCO 형식)으로 학습·평가한다 ④ 실습 코드를 리뷰한다
진행 방식	과제 원본 코드를 순서대로 실행하며 각 코드 아래에 의미를 정리 하고, 고칠 점이 있는 코드는 바로 아래에 🔍 코드 리뷰 (문제 → 영향 → 개선안)를 붙임
환경	Windows 11 · Python 3.11 · PyTorch 2.11 (CUDA 12.8) · RTX 5060 Laptop 8GB · Yet-Another-EfficientDet-Pytorch
코드 표기	원본은 Colab용이다. 이 PC에서 그대로 안 되는 줄만 코드 안에 # 코랩용 (원본) 과 # 노트북용 을 나란히 적었고, 원본에 없는 셀은 첫 줄에 # [추가] 라고 적었다

목차

- Part 1. 사전학습 모델로 이미지 추론 (🔍 코드 리뷰 1-1 ~ 1-6)
- Part 2. 영상 분석 (🔍 코드 리뷰 2-1 ~ 2-7)
- Part 3. Custom dataset 학습 (차량) (🔍 코드 리뷰 3-1 ~ 3-12, 리뷰 근거 확인)
- 코드 리뷰 종합 · 목표 기준 평가 · 진행하면서 막혔던 부분과 해결 · 개인 회고

```
In [1]: # [추가] 노트북용 셀 (원본에 없음): 커널이 .venv 파이썬이고 GPU가 보이는지 확인
import torch, sys
print("torch", torch.__version__, "| GPU 사용 가능:", torch.cuda.is_available(),
      "|", torch.cuda.get_device_name(0))
print("파이썬 위치:", sys.executable)
```

torch 2.11.0+cu128 | GPU 사용 가능: True | NVIDIA GeForce RTX 5060 Laptop GPU
파이썬 위치: C:\Users\LQ05060\Desktop\cv-4\.venv\Scripts\python.exe

Part 1. 사전학습 모델로 이미지 추론

EfficientDet 실습

1. 라이브러리 설치 및 환경 설정

```
In [2]: !git clone https://github.com/zylo117/Yet-Another-EfficientDet-Pytorch.git
# !pip install opencv-python matplotlib pycocotools
```

EfficientDet을 PyTorch로 구현한 오픈소스 레포를 받는다. ultralytics처럼 pip 로 설치하는 패키지가 아니라, 레포 안의 `backbone.py` , `utils/` 같은 파이썬 파일을 직접 import 해서 쓰는 방식이다. 이 PC에는 레포를 미리 받아 두어서 `fatal: destination path ... already exists` 가 출력된다. 이미 있다는 뜻이라 그대로 진행하면 된다.

```
In [3]: import os

os.chdir('Yet-Another-EfficientDet-Pytorch')
print("현재 디렉토리:", os.getcwd())
```

현재 디렉토리: C:\Users\LQ05060\Desktop\cv-4\Yet-Another-EfficientDet-Pytorch

작업 폴더를 레포 안으로 옮긴다. 그래야 `from backbone import ...` 같은 import와 `weights/` , `test/` 같은 상대 경로가 동작한다. 이후 모든 경로는 레포 폴더 기준이다.

```
In [4]: # 🐛 코랩용 (원본)
# !mkdir -p weights
# !wget https://github.com/zylo117/Yet-Another-Efficient-Pytorch/releases/download/1.0/efficientdet-d0.pth -O weights/efficientdet-d0.pth

# 📖 노트북용: Windows 명령창의 mkdir은 -p를 몰라 '-p' 폴더를 따로 만들고, wget
# 은 없어서 Windows에 기본으로 있는 curl로 받는다
!if not exist weights mkdir weights
!curl -L https://github.com/zylo117/Yet-Another-Efficient-Pytorch/releases/download/1.0/efficientdet-d0.pth -o weights/efficientdet-d0.pth
```

COCO 데이터로 미리 학습된 EfficientDet-D0 가중치(약 15MB)를 `weights/` 폴더에 받는다. D0는 D0~D7 중 가장 작고 빠른 모델이다. 주소의 레포 이름이 `Yet-Another-Efficient-Pytorch` (Det가 빠짐)인데, 오타가 아니라 실제로 가중치가 올라가 있는 곳이다.

📖 내 컴퓨터에서 돌리려고 바꾼 곳 — 원본의 `!mkdir -p` 와 `!wget` 은 Windows 명령창에서 동작하지 않아 `if not exist ... mkdir` 과 Windows 내장 `curl` 로 바꿨다. 받는 주소와 저장 위치는 원본과 같다.

```
In [5]: import torch
from backbone import EfficientDetBackbone
compound_coef = 0
model = EfficientDetBackbone(compound_coef=0, num_classes=90)
```

모델의 뼈대(구조)만 만든다. 아직 가중치는 무작위 값이다.

- `compound_coef` : D0~D7 중 몇 번 모델인지. 숫자가 클수록 입력 크기·층 수·채널 수가 함께 커진다(Compound Scaling).
- `num_classes=90` : COCO 클래스 번호가 1~90까지 있어서 90이다. 실제 물체는 80종이고 빈 번호가 10개 있다.

<code>compound_coef</code>	모델	입력 크기	파라미터	COCO mAP
0	D0	512×512	3.9M	33.8
1	D1	640×640	6.6M	39.6
3	D3	896×896	12M	47.5
7	D7	1536×1536	가장 큼	가장 높음

🔍 코드 리뷰 1-1 · `compound_coef` 변수를 쓰지 않음

- 문제: `compound_coef = 0` 변수를 만들어 놓고, 모델은 `compound_coef=0` 처럼 숫자를 직접 넣어 만든다
- 영향: 변수를 D1로 바꿔도 모델은 D0로 만들어져, 입력 크기(640)와 모델이 어긋난다
- 개선안: `EfficientDetBackbone(compound_coef=compound_coef, ...)`

```
model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=90)
```

```
In [6]: model.load_state_dict(torch.load(f'weights/efficientdet-d0.pth', map_location='cpu'))
model.requires_grad_(False)
model.eval()
```

```

Out[6]: EfficientDetBackbone(
  (bifpn): Sequential(
    (0): BiFPN(
      (conv6_up): SeparableConvBlock(
        (depthwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), groups=64, bias=False)
        )
        (pointwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1))
        )
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.01, affine=True, track_running_stats=True)
      )
      (conv5_up): SeparableConvBlock(
        (depthwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), groups=64, bias=False)
        )
        ... (627줄 생략) ...
        (_se_expand): Conv2dStaticSamePadding(
          (conv): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
        )
        (_project_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(1152, 320, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
        (_bn2): BatchNorm2d(320, eps=0.001, momentum=0.010000000000000009, affine=True, track_running_stats=True)
        (_swish): MemoryEfficientSwish()
      )
      (_swish): MemoryEfficientSwish()
    )
  )
)

```

받아 둔 가중치를 뼈대의 각 층에 이름표대로 채워 넣는다(`load_state_dict`). `map_location='cpu'` 는 GPU에서 저장된 파일이어도 일단 CPU 메모리로 읽어 오라는 뜻이다. 학습을 하지 않으니 `requires_grad_(False)` 로 기울기 계산을 끄고, `eval()` 로 BatchNorm-Dropout을 추론 모드로 바꾼다. 출력된 긴 구조는 크게 `bifpn` (여러 크기 특징 섞기), `regressor` (박스 위치 예측), `classifier` (클래스 예측), `backbone_net` (EfficientNet 특징 추출) 네 덩어리다.

```
In [7]: compound_coef = 0
force_input_size = None # set None to use default size
# img_path = 'bus.jpg'

# replace this part with your project's anchor config
anchor_ratios = [(1.0, 1.0), (1.4, 0.7), (0.7, 1.4)]
anchor_scales = [2 ** 0, 2 ** (1.0 / 3.0), 2 ** (2.0 / 3.0)]

threshold = 0.2
iou_threshold = 0.2

use_cuda = True
use_float16 = False
```

추론 설정값을 정한다.

- **앵커(anchor):** 모델이 박스를 예측할 때 기준으로 삼는 "미리 깔아 둔 박스"다. 모델은 박스를 처음부터 그리지 않고, 앵커를 얼마나 옮기고 늘릴지만 예측한다.
 - `anchor_ratios` : 앵커의 (가로, 세로) 배율 3종 → 정사각형, 가로로 긴 것, 세로로 긴 것
 - `anchor_scales` : 크기 3종 → 1, 1.26, 1.59배. 그래서 특징맵 한 칸마다 $3 \times 3 = 9$ 개 앵커가 생긴다.
- `threshold = 0.2` : 클래스 점수가 0.2보다 낮은 예측은 버린다(YOLO의 `conf`).
- `iou_threshold = 0.2` : NMS 기준. 같은 클래스 박스끼리 IoU가 0.2보다 많이 겹치면 중복으로 보고 하나만 남긴다. 낮을수록 많이 지워서 붙어 있는 물체를 놓칠 수 있다.
- `use_float16` : 16비트(반정밀도) 연산 여부

🔍 코드 리뷰 1-2 · 앵커 설정이 모델에 전달되지 않음

- **문제:** `anchor_ratios`, `anchor_scales` 를 정의만 하고 모델에 넘기지 않는다
- **영향:** 지금은 레포 기본값과 같아서 결과가 같지만, 값을 바꿔도 반영되지 않는다. Part 3의 앵커 불일치 문제와 같은 원인이다
- **개선안:** 모델 생성 시 `ratios=anchor_ratios, scales=anchor_scales`

```
# 앵커 설정 셀을 모델 생성보다 위로 올리고
model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=9
0,
                             ratios=anchor_ratios, scales=anchor_scales)
```

```
In [8]: obj_list = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light',
                   'fire hydrant', '', 'stop sign', 'parking meter', 'bench', 'bird',
                   'cat', 'dog', 'horse', 'sheep',
                   'cow', 'elephant', 'bear', 'zebra', 'giraffe', '', 'backpack', 'umbrella', '', '', 'handbag', 'tie',
                   'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite',
                   'baseball bat', 'baseball glove',
                   'skateboard', 'surfboard', 'tennis racket', 'bottle', '', 'wine glasses', 'cup', 'fork', 'knife', 'spoon',
                   'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut',
                   'cake', 'chair', 'couch', 'potted plant', 'bed', '', 'dining table',
                   '', '', 'toilet', '', 'tv',
                   'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave',
                   'oven', 'toaster', 'sink',
                   'refrigerator', '', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier',
                   'toothbrush']
```

```
In [9]: len(obj_list)
```

```
Out[9]: 90
```

obj_list 는 리스트 위치 = 클래스 번호다. 중간의 '' 는 COCO에서 비어 있는 번호 자리로, 이 칸까지 채워야 모델이 내놓는 번호와 이름이 맞는다. 길이 90을 확인했다.

```
In [10]: from PIL import Image
         Image.open('test/img.png')
```

```
Out[10]:
```



레포에 들어 있는 1920×1080 거리 사진을 테스트 이미지로 쓴다.

```
In [11]: from utils.utils import preprocess, invert_affine, postprocess, STANDARD_COLORS,
standard_to_bgr, get_index_label, plot_one_box

img_path = 'test/img.png'
color_list = standard_to_bgr(STANDARD_COLORS)
input_sizes = [512, 640, 768, 896, 1024, 1280, 1280, 1536, 1536]
input_size = input_sizes[compound_coef] if force_input_size is None else force_i
nput_size
ori_imgs, framed_imgs, framed metas = preprocess(img_path, max_size=input_size)
if use_cuda:
    x = torch.stack([torch.from_numpy(fi).cuda() for fi in framed_imgs], 0)
else:
    x = torch.stack([torch.from_numpy(fi) for fi in framed_imgs], 0)
x = x.to(torch.float32 if not use_float16 else torch.float16).permute(0, 3, 1,
2)
```

이미지를 모델 입력 모양 (1, 3, 512, 512) 텐서로 바꾼다. `preprocess` 안에서는 다음 일이 일어난다.

1. `cv2.imread` 로 읽는다(BGR 순서).
2. BGR → RGB로 뒤집고 `/255` 로 0~1로 만든 뒤, ImageNet 평균·표준편차로 정규화한다.
3. `aspectaware_resize_padding` : 비율을 유지해 긴 변을 512로 줄이고(1920×1080 → 512×288), 남은 아래쪽을 0으로 채워 512×512를 만든다.

`torch.stack(..., 0)` 은 이미지 리스트를 배치 차원으로 쌓고, `permute(0, 3, 1, 2)` 는 (배치, 높이, 너비, 채널) 을 PyTorch가 쓰는 (배치, 채널, 높이, 너비) 로 바꾼다.

```
In [12]: ori_imgs[0].shape, framed_imgs[0].shape
```

```
Out[12]: ((1080, 1920, 3), (512, 512, 3))
```

```
In [13]: framed_metas
```

```
Out[13]: [(512, 288, 1920, 1080, 0, 224)]
```

원본은 (1080, 1920, 3), 모델 입력은 (512, 512, 3)이다. `framed_metas` 는 전처리에서 바꾼 내용을 기록한 값으로 (new_w, new_h, old_w, old_h, padding_w, padding_h) = (512, 288, 1920, 1080, 0, 224) 다. 아래쪽 패딩 224 = 512 - 288이다. YOLO의 letterbox는 이미지를 가운데 두고 위아래에 패딩을 넣지만, 여기는 왼쪽 위에 붙이고 오른쪽·아래만 채운다. 그래서 나중에 좌표를 되돌릴 때 패딩을 빼지 않고 배율만 나누면 된다.

```
In [14]: if use_cuda:
    model = model.cuda()
if use_float16:
    model = model.half()
with torch.no_grad():
    features, regression, classification, anchors = model(x)
```

```
In [15]: features[0].shape, regression[0].shape, classification.shape, anchors.shape
```


49104개 앵커 예측을 사람이 읽을 수 있는 박스로 바꾼다.

- `BBoxTransform` : 앵커 + regression $\rightarrow$ 실제 박스 `(x1, y1, x2, y2)`. 중심은 앵커 중심 + $(dy, dx) \times$ 앵커 크기, 크기는 앵커 크기 $\times \exp(dh, dw)$
- `ClipBoxes` : 이미지(512×512) 밖으로 나간 좌표를 경계에 맞춰 자른다.
- `postprocess` : 앵커마다 가장 높은 클래스 점수가 `threshold` 를 넘는 것만 남기고, 클래스별 NMS(`batched_nms`)로 겹치는 박스를 지운다.

🔍 코드 리뷰 1-3 · 같은 추론을 두 번 실행

- **문제**: 바로 앞 셀에서 돌린 `model(x)` 를 한 번 더 돌린다
- **영향**: 같은 결과를 얻으려고 추론을 두 번 한다
- **개선안**: 앞 셀의 `regression`, `classification`, `anchors` 를 그대로 쓴다

```
with torch.no_grad():
    features, regression, classification, anchors = model(x)  # 추론은
    한 번만
out = postprocess(x, anchors, regression, classification,
                  BBoxTransform(), ClipBoxes(), threshold, iou_threshol
d)
```

In [19]: out

```
Out[19]: [{'rois': array([[333.9106      , 239.92058      , 424.38052      , 287.47403      ],
                          [314.5778      , 129.69664      , 351.32883      , 163.34978      ],
                          [350.18918      , 165.4527      , 433.36374      , 241.53989      ],
                          [130.6782      , 232.14676      , 207.67924      , 287.59213      ],
                          [362.0041      , 135.1465      , 408.65985      , 169.68913      ],
                          [ 56.523476    , 167.49417      , 129.53165      , 224.73514      ],
                          [414.8751      , 173.18457      , 506.78836      , 242.55646      ],
                          [  0.8899746    , 233.43506      ,  48.700165     , 277.26663      ],
                          [ 80.5062      , 246.80457      , 122.74743      , 286.69556      ],
                          [232.98915      , 165.0869      , 285.26852      , 247.47136      ],
                          [155.37982      ,  93.800156     , 196.91263      , 136.37265      ],
                          [330.41754      , 184.252      , 350.99792      , 229.17503      ],
                          [290.06723      , 180.9333      , 309.02994      , 222.60262      ],
                          [236.92595      , 112.800186    , 263.35764      , 136.59938      ],
                          [316.6743      ,  98.80613      , 343.58554      , 120.447014     ],
                          ... (18줄 생략) ...
                          [209.94847      ,  61.93464      , 231.8347      ,  76.26406      ],
                          [234.78328      , 140.73515      , 253.98491      , 167.97643      ],
                          [180.91132      , 200.99048      , 199.90353      , 231.18988      ],
                          [103.97833      , 197.53981      , 129.49454      , 237.01988      ]]),
            dtype=float32),
          'class_ids': array([2, 2, 2, 2, 2, 2, 2, 3, 3, 2, 5, 0, 3, 2, 2, 3, 0, 3, 0,
                              2, 0, 0,
                              3, 5, 0, 2, 0, 2, 2, 0, 2, 3, 2, 2, 2, 3, 0], dtype=int64),
          'scores': array([0.846261    , 0.6979735   , 0.6890693   , 0.6842498   , 0.680701    ,
                           0.6492652   , 0.6223136   , 0.48397562  , 0.44831446  , 0.43913242  ,
                           0.41518062  , 0.4151232   , 0.41261926  , 0.398776   , 0.3957473   ,
                           0.37491718  , 0.36878547  , 0.36397147  , 0.3620192   , 0.35451776  ,
                           0.30066815  , 0.29696298  , 0.25586146  , 0.23827921  , 0.23651804  ,
                           0.22857411  , 0.22554037  , 0.22411102  , 0.21962756  , 0.219046    ,
                           0.2129027   , 0.21227047  , 0.21129072  , 0.20681176  , 0.20218116  ,
                           0.20184788  , 0.20113276], dtype=float32)}}]
```

```
In [20]: out[0]['rois'].shape, out[0]['class_ids'].shape, out[0]['scores'].shape
```

```
Out[20]: ((37, 4), (37,), (37,))
```

결과는 이미지마다 {'rois': 박스, 'class_ids': 클래스 번호, 'scores': 점수} 딕셔너리다. 49104개 앵커가 37개 박스로 줄었다. 이 좌표는 아직 512×512 입력 기준이다.

```
In [21]: import cv2
def display(preds, imgs, imshow=True, imwrite=False):
    for i in range(len(imgs)):
        if len(preds[i]['rois']) == 0:
            continue

        imgs[i] = imgs[i].copy()

        for j in range(len(preds[i]['rois'])):
            x1, y1, x2, y2 = preds[i]['rois'][j].astype(np.int32)
            obj = obj_list[preds[i]['class_ids'][j]]
            score = float(preds[i]['scores'][j])
            plot_one_box(imgs[i], [x1, y1, x2, y2], label=obj, score=score, color
                        =color_list[get_index_label(obj, obj_list)])

        if imshow:
            cv2.imshow('img', imgs[i])
            cv2.waitKey(0)

        if imwrite:
            cv2.imwrite(f'test/img_inferred_d{compound_coef}_this_repo_{i}.jpg',
                        imgs[i])

    return f'test/img_inferred_d{compound_coef}_this_repo_'
```

결과를 그리는 함수다. 박스마다 좌표를 정수로 바꾸고, class_ids 로 obj_list 에서 이름을 찾아 레포 유틸 plot_one_box 로 박스와 라벨을 그린다. 색은 클래스 이름별로 정해진다. imshow=True 면 OpenCV 창을 띄우고, imwrite=True 면 test/ 폴더에 파일로 저장한다. 반환값은 저장 파일 이름의 앞부분이다.

🔍 코드 리뷰 1-4 · 함수보다 늦게 나오는 import

- **문제:** 함수 안에서 np 를 쓰는데 import numpy as np 는 다음 셀에 있다
- **영향:** 파이썬은 호출할 때 이름을 찾아서 지금은 동작하지만, 셀 순서가 바뀌면 NameError 가 난다
- **개선안:** import는 맨 위에 모은다

```
In [22]: import numpy as np
out = invert_affine(framed metas, out)
img_path = display(out, ori_imgs, imshow=False, imwrite=True)
```

```
In [23]: img_path
```

```
Out[23]: 'test/img_inferred_d0_this_repo_'
```

`invert_affine` 이 512 기준 좌표를 원본 1920×1080 기준으로 되돌린다. x는 512/1920, y는 288/1080으로 나눈다(줄인 배율만큼 다시 키우기). 그다음 원본 이미지에 그려 파일로 저장했다.

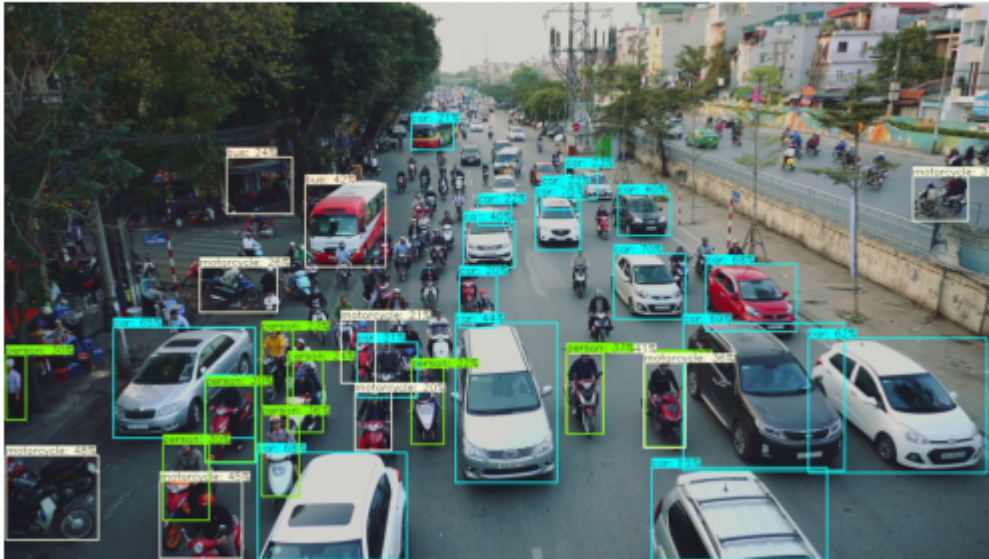
```
In [24]: from PIL import Image
import matplotlib.pyplot as plt

img_path = f"{img_path}0.jpg" # 파일 확장자에 따라 수정 가능

img = Image.open(img_path)

# 이미지 출력
plt.imshow(img)
plt.axis('off') # 축 제거
plt.title(img_path)
plt.show()
```

test/img_inferred_d0_this_repo_0.jpg



저장된 결과를 띄웠다. 거리의 차와 사람에 박스가 그려졌다. 파일로 저장할 때 OpenCV가 BGR을 처리하고 PIL이 RGB로 읽어서 색이 정상으로 보인다.

🔍 코드 리뷰 1-5 · 한 변수를 여러 뜻으로 재사용

- **문제:** `img_path` 한 변수를 입력 경로 → 저장 이름 앞부분 → 저장 파일 경로로 바꿔 쓴다
- **영향:** 셀을 다시 실행하면 `...this_repo_0.jpg0.jpg` 처럼 경로가 꼬인다
- **개선안:** `save_prefix`, `result_path` 처럼 이름을 나눈다

```
save_prefix = display(out, ori_imgs, imshow=False, imwrite=True)
result_path = f"{save_prefix}0.jpg"
img = Image.open(result_path)
```

🔍 코드 리뷰 1-6 · 같은 추론 코드가 파트마다 반복

- **문제:** 전처리 → 추론 → 후처리 → 좌표 복원 → 그리기를 셀마다 풀어 썼고, Part 2:3 에서 같은 코드를 또 쓴다
- **영향:** 같은 코드가 세 번 반복되고, 한 곳만 고치면 나머지가 어긋난다
- **개선안:** `detect(model, img, obj_list, threshold)` 함수 하나로 묶는다

```
def detect(model, img_path, obj_list, threshold=0.2, iou_threshold=0.2,
max_size=512):
    ori, framed, metas = preprocess(img_path, max_size=max_size)
    x = torch.stack([torch.from_numpy(f).cuda() for f in framed], 0).float().permute(0, 3, 1, 2)
    with torch.no_grad():
        _, reg, cls, anc = model(x)
        out = postprocess(x, anc, reg, cls, BBoxTransform(), ClipBoxes(), threshold, iou_threshold)
    return ori, invert_affine(metas, out)
```

Part 2. 영상 분석

영상은 이미지의 연속이라, 프레임마다 Part 1과 같은 과정을 반복한다.

영상 분석

```
In [25]: # 🐞 코랩용 (원본)
# !pwd
# 📖 노트북용: Windows 명령창에는 pwd가 없다 → 파이썬으로 현재 폴더를 본다
os.getcwd()
```

```
Out[25]: 'C:\\Users\\LQ5060\\Desktop\\cv-4\\Yet-Another-EfficientDet-Pytorch'
```

```
In [26]: # 🐞 코랩용 (원본)
# os.path.exists('soccer.mp4')
# 📖 노트북용: 받은 영상 파일은 soccer.gif (원본 CoLab 실행 기록에도 이 셀은 False였다)
os.path.exists('soccer.gif')
```

```
Out[26]: True
```

영상 파일이 있는지 확인한다. 과제 자료로 받은 영상은 `soccer.gif` (2732×1440, 367 프레임)다. 원본 Colab 실행 기록에서 `soccer.mp4` 확인 결과가 `False` 였고 결과 프레임 크기가 2732×1440이었으니, 원래 실행에서도 이 gif를 썼던 것으로 보인다.

내 컴퓨터에서 돌리려고 바꾼 곳

- 원본의 `soccer.mp4` 대신 받은 `soccer.gif` 를 확인한다.
- `!pwd` → `os.getcwd()` (Windows 명령창에는 `pwd`가 없다).

코드 리뷰 2-1 · 영상 확인 결과를 쓰지 않음

- **문제:** `os.path.exists('soccer.mp4')` 로 확인만 하고, `False` 여도 그대로 진행한다
- **영향:** 원본 Colab에서도 `False` 였다. 파일이 없으면 `cap.read()` 가 바로 실패해 프레임이 0장 저장되고, 다음 단계(`images[0]`)에서야 `IndexError` 가 난다
- **개선안:** 영상이 열렸는지 `cap.isOpened()` 로 확인하고, 실패하면 바로 멈춘다

```
cap = cv2.VideoCapture(video_src)
if not cap.isOpened():
    raise FileNotFoundError(f'영상을 열 수 없습니다: {video_src}')
```

```
In [27]: # Core Author: Zylo117
         # Script's Author: winter2897
```

```

"""
Simple Inference Script of EfficientDet-Pytorch for detecting objects on webcam
"""

import time
import torch
import cv2
import numpy as np
from torch.backends import import cudnn
from backbone import EfficientDetBackbone
from efficientdet.utils import import BBoxTransform, ClipBoxes
from utils.utils import import preprocess, invert_affine, postprocess, preprocess_video

# Video's path
# 🐼 코랩용 (원본)
# video_src = 'soccer.mp4' # set int to use webcam, set str to read from a video file
# 📺 노트북용: 받은 영상 파일은 soccer.gif (OpenCV는 gif도 영상처럼 읽는다)
video_src = 'soccer.gif' # set int to use webcam, set str to read from a video file
# output_dir = './output_soccer_frames'
# 🐼 코랩용 (원본)
# output_dir = '/content/Yet-Another-EfficientDet-Pytorch/output_soccer_frames'
# output_video_path = '/content/Yet-Another-EfficientDet-Pytorch/output_soccer.mp4'
# 📺 노트북용: /content/는 Colab에만 있는 폴더라 레포 기준 상대 경로로
output_dir = './output_soccer_frames'
output_video_path = './output_soccer.mp4'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

compound_coef = 0
force_input_size = 512 # set None to use default size

threshold = 0.2
iou_threshold = 0.2

use_cuda = True
use_float16 = False

obj_list = ['person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light',
            'fire hydrant', '', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep',
            'cow', 'elephant', 'bear', 'zebra', 'giraffe', '', 'backpack', 'umbrella', '', '', 'handbag', 'tie',
            'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove',
            'skateboard', 'surfboard', 'tennis racket', 'bottle', '', 'wine glass', 'cup', 'fork', 'knife', 'spoon',
            'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut',
            'cake', 'chair', 'couch', 'potted plant', 'bed', '', 'dining table', '', '', 'toilet', '', 'tv',
            'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink',
            'refrigerator', '', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toys']

```

```

r', 'hair drier',
    'toothbrush']

input_sizes = [512, 640, 768, 896, 1024, 1280, 1280, 1536, 1536]
input_size = input_sizes[compound_coef] if force_input_size is None else force_input_size

# Load model
model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=len(obj_list))
model.load_state_dict(torch.load(f'weights/efficientdet-d{compound_coef}.pth'))
model.requires_grad_(False)
model.eval()

```

```

Out[27]: EfficientDetBackbone(
  (bifpn): Sequential(
    (0): BiFPN(
      (conv6_up): SeparableConvBlock(
        (depthwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), groups=64, bias=False)
        )
        (pointwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1))
        )
        (bn): BatchNorm2d(64, eps=0.001, momentum=0.01, affine=True, track_running_stats=True)
      )
      (conv5_up): SeparableConvBlock(
        (depthwise_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), groups=64, bias=False)
        )
        ... (627줄 생략) ...
        (_se_expand): Conv2dStaticSamePadding(
          (conv): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
        )
        (_project_conv): Conv2dStaticSamePadding(
          (conv): Conv2d(1152, 320, kernel_size=(1, 1), stride=(1, 1), bias=False)
        )
        (_bn2): BatchNorm2d(320, eps=0.001, momentum=0.010000000000000009, affine=True, track_running_stats=True)
        (_swish): MemoryEfficientSwish()
      )
      (_swish): MemoryEfficientSwish()
    )
  )
)

```

레포의 영상 추론 스크립트(`efficientdet_test_videos.py`)를 그대로 옮긴 셀이다. 영상 경로와 결과 저장 위치를 정하고, 설정값과 `obj_list` 를 다시 정의한 뒤 모델을 처음부터 다시 만들어 가중치를 불러온다. `video_src` 가 문자열이면 영상 파일을, 정수 (`0`)면 웹캠을 연다.

 **내 컴퓨터에서 돌리려고 바꾼 곳** — `video_src` 를 `soccer.gif` 로, Colab 전용 경로(`/content/...`)를 레포 기준 상대 경로로 바꿨다. 나머지는 원본 그대로다.

🔍 코드 리뷰 2-2 · 모델 설정을 통째로 다시 정의

- **문제:** Part 1에서 만든 모델과 `obj_list` , 설정값을 통째로 다시 정의한다
- **영향:** 스크립트를 그대로 붙여 넣어 생긴 중복이다. 한쪽만 고치면 값이 어긋난다
- **개선안:** Part 1의 모델을 재사용하거나 설정을 한 곳에 모은다

```
In [28]: if use_cuda:
          model = model.cuda()
if use_float16:
          model = model.half()
```

```
In [29]: # function for display
def display(preds, imgs):
    for i in range(len(imgs)):
        if len(preds[i]['rois']) == 0:
            return imgs[i]

        for j in range(len(preds[i]['rois'])):
            (x1, y1, x2, y2) = preds[i]['rois'][j].astype(np.int64)
            cv2.rectangle(imgs[i], (x1, y1), (x2, y2), (255, 255, 0), 2)
            obj = obj_list[preds[i]['class_ids'][j]]
            score = float(preds[i]['scores'][j])

            cv2.putText(imgs[i], '{}', {:.3f}.format(obj, score),
                        (x1, y1 + 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                        (255, 255, 0), 1)

        return imgs[i]

# Box
regressBoxes = BBoxTransform()
clipBoxes = ClipBoxes()
```

영상용 `display` 는 `plot_one_box` 대신 `cv2.rectangle` (박스)과 `cv2.putText` (이름·점수)로 직접 그린다. 색 (255, 255, 0) 은 OpenCV가 BGR 순서라 하늘색이다. `regressBoxes` , `clipBoxes` 는 루프 밖에서 한 번만 만들어 둔다.

🔍 코드 리뷰 2-3 · 반복문 안의 `return`

- **문제:** 반복문 안에서 `return` 한다
- **영향:** 지금은 프레임이 한 장씩이라 문제없지만, 여러 장을 한 번에 넣으면 첫 장만 처리하고 끝난다
- **개선안:** `return` 을 반복문 밖으로 빼고 이미지 리스트를 돌려준다

```
def display(preds, imgs):  
    for i in range(len(imgs)):  
        for j in range(len(preds[i]['rois'])):  
            ... # 박스·글자 그리기  
    return imgs # 반복문이 다 끝난 뒤에 돌려준다
```

```

In [30]: # Video capture
cap = cv2.VideoCapture(video_src)
frame_idx = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # frame preprocessing
    ori_imgs, framed_imgs, framed metas = preprocess_video(frame, max_size=input
_size)

    if use_cuda:
        x = torch.stack([torch.from_numpy(fi).cuda() for fi in framed_imgs], 0)
    else:
        x = torch.stack([torch.from_numpy(fi) for fi in framed_imgs], 0)

    x = x.to(torch.float32 if not use_float16 else torch.float16).permute(0, 3,
1, 2)

    # model predict
    with torch.no_grad():
        features, regression, classification, anchors = model(x)

        out = postprocess(x,
                        anchors, regression, classification,
                        regressBoxes, clipBoxes,
                        threshold, iou_threshold)

    # result
    out = invert_affine(framed metas, out)
    img_show = display(out, ori_imgs)

    # show frame by frame
    img_show = display(out, ori_imgs)

    # 프레임 저장
    frame_path = os.path.join(output_dir, f'frame_{frame_idx:05d}.jpg')
    cv2.imwrite(frame_path, img_show)
    frame_idx += 1

cap.release()
# cv2.destroyAllWindows()

```

영상을 한 프레임씩 읽어(`cap.read()`), 끝나면(`ret` 이 `False`) 멈춘다. 프레임마다 `p`
`reprocess_video` (경로 대신 프레임 배열을 받는 전처리) → 모델 → `postprocess` →
`invert_affine` → `display` 를 거쳐, 박스를 그린 프레임을 `frame_00000.jpg` 처럼
다섯 자리 번호로 저장한다. 번호를 0으로 채워야 이름순 정렬이 곧 시간순이 된다. RTX
5060에서 367프레임을 처리하는 데 3~4분 걸렸다.

🔍 코드 리뷰 2-4 · `display` 를 두 번 호출

- **문제:** `display` 를 두 번 호출한다
- **영향:** 같은 프레임에 박스와 글자가 두 번 겹쳐 그려진다(같은 자리라 눈에 잘 안 띈다)
- **개선안:** 한 번만 호출한다

```
out = invert_affine(framed metas, out)
img_show = display(out, ori_imgs)  # 한 번만
```

🔍 코드 리뷰 2-5 · `preprocess_video` 의 평균·표준편차 순서

- **문제:** 기본 평균·표준편차가 `(0.406, 0.456, 0.485)` 로 `preprocess` 의 `(0.485, 0.456, 0.406)` 과 순서가 반대인데, 둘 다 똑같이 BGR → RGB로 뒤집은 뒤 뺀다
- **영향:** 영상에서는 R과 B 채널에 평균이 바뀌어 적용된다. 이미지 추론과 입력 분포가 조금 달라진다
- **개선안:** `preprocess` 와 같은 평균·표준편차를 넘긴다

```
ori_imgs, framed_imgs, framed_metas = preprocess_video(
    frame, max_size=input_size, mean=(0.485, 0.456, 0.406), std=(0.229,
0.224, 0.225))
```

```
In [31]: # 🐞 코랩용 (원본)
# !ls -l
# 📁 노트북용: Windows 명령창에는 ls가 없다 → 파이썬으로 폴더 내용을 본다
for name in sorted(os.listdir('.')):
    print(name)
```

```
.git
.gitignore
LICENSE
__pycache__
backbone.py
benchmark
coco_eval.py
datasets
efficientdet
efficientdet_test.py
efficientdet_test_videos.py
efficientnet
output_soccer.mp4
output_soccer_frames
projects
readme.md
res
soccer.gif
test
train.py
tutorial
utils
weights
```

```
In [32]: os.listdir(output_dir)[:10]
```

```
Out[32]: ['frame_00000.jpg',
          'frame_00001.jpg',
          'frame_00002.jpg',
          'frame_00003.jpg',
          'frame_00004.jpg',
          'frame_00005.jpg',
          'frame_00006.jpg',
          'frame_00007.jpg',
          'frame_00008.jpg',
          'frame_00009.jpg']
```

레포 폴더에 `output_soccer_frames` 와 `output_soccer.mp4` 가 생겼다. `os.listdir` 는 이름순으로 정렬해 주지 않아서 프레임 번호가 뒤섞여 나온다.

 내 컴퓨터에서 돌리려고 바꾼 곳 — `!ls -l` → `os.listdir`
(Windows 명령창에는 ls가 없다).

```
In [33]: img = Image.open(os.path.join(output_dir, 'frame_00004.jpg'))
img
```

Out[33]:



5번째 프레임에서는 사람 7명(`person` , 신뢰도 0.28~0.89), 벤치 3개(`bench`), 뒤쪽 차 1대(`car`)를 잡았다. 축구공은 COCO에 `sports ball` 클래스가 있는데도 잡지 못했다. 2732픽셀 너비를 512로 줄이면 공 지름이 약 16픽셀로 작아지기 때문으로 보인다. 프레임 크기는 원본과 같은 2732×1440이다.

```
In [34]: # 📁 이미지들을 비디오로 합치기
images = sorted([img for img in os.listdir(output_dir) if img.endswith(".jpg")])
first_frame = cv2.imread(os.path.join(output_dir, images[0]))
height, width, _ = first_frame.shape

fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out_video = cv2.VideoWriter(output_video_path, fourcc, 20.0, (width, height)) # 20fps

for image_file in images:
    img = cv2.imread(os.path.join(output_dir, image_file))
    out_video.write(img)

out_video.release()
print(f"✅ 영상 생성 완료: {output_video_path}")

✅ 영상 생성 완료: ./output_soccer.mp4
```

```
In [35]: os.path.exists(output_video_path)
```

```
Out[35]: True
```

저장한 프레임을 `sorted` 로 순서대로 정렬해 mp4로 합친다. 첫 프레임에서 가로·세로 크기를 얻고, `VideoWriter` 에 mp4 코덱(`mp4v`)과 초당 20프레임을 지정해 한 장씩 쓴 뒤 `release()` 로 파일을 마무리한다.

🔍 코드 리뷰 2-6 · fps 고정

- **문제:** fps를 20으로 고정했다
- **영향:** soccer.gif 는 약 23fps라 결과 영상이 원본보다 느리게 재생된다
- **개선안:** cap.get(cv2.CAP_PROP_FPS) 로 원본 fps를 읽어 쓴다

```
fps = cv2.VideoCapture(video_src).get(cv2.CAP_PROP_FPS) # soccer.gif는  
약 23  
out_video = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height))
```

🔍 코드 리뷰 2-7 · 프레임을 모두 저장했다가 다시 합치기

- **문제:** 2732×1440 프레임 367장을 jpg로 모두 저장했다가 다시 읽어 영상으로 만든다
- **영향:** 디스크와 시간이 두 번 든다. 이 PC에서는 프레임 폴더와 mp4(약 360MB)가 따로 생겼다
- **개선안:** 루프 안에서 바로 VideoWriter 에 쓴다

```
writer = None  
while True:  
    ...  
    if writer is None:  
        h, w = img_show.shape[:2]  
        writer = cv2.VideoWriter(output_video_path, cv2.VideoWriter_fourcc(*'mp4v'), fps, (w, h))  
        writer.write(img_show) # jpg로 저장하지 않고 바로 영상에 쓴다  
    writer.release()
```

Part 3. Custom dataset 학습 (차량)

학습

Custom dataset 학습 (Car video)

<https://www.kaggle.com/datasets/pkdarabi/vehicle-detection-image-dataset>

1. 데이터셋 준비

- Yet-Another-EfficientDet-Pytorch
 - datasets
 - car_detect
 - Apply_Grayscale
 - Apply_Grayscale
 - Vehicles_Detection.v9i.coco
 - test
 - _annotations.coco.json
 - XXXX.jpg
 - train
 - valid
- Yet-Another-EfficientDet-Pytorch
 - projects
 - my_car_detect_proj.yml
 - datasets
 - car_detect
 - my_car_detect_proj
 - annotations
 - instances_train.json
 - instances_valid.json
 - instances_test.json
 - train
 - XXX.jpg
 - XXX.jpg
 - valid
 - XXX.jpg
 - XXX.jpg
 - test
 - XXX.jpg
 - XXX.jpg

Kaggle 차량 탐지 데이터셋은 Roboflow에서 **COCO 형식**으로 내보낸 것으로, 흑백 버전 (`Apply_Grayscale` , v9i)과 컬러 버전(`No_Apply_Grayscale` , v8i)이 들어 있다. 위가 받은 그대로의 구조, 아래가 레포의 `train.py` 가 읽는 구조다. `archive.zip` 은 미리 `datasets/` 폴더에 넣어 두었다.

포맷	라벨 파일	좌표 형식
COCO (이번 실습)	JSON 하나에 전체 이미지 라벨	x, y, width, height (픽셀, 왼쪽 위 기준)
VOC	이미지마다 XML	xmin, ymin, xmax, ymax
YOLO (지난 실습)	이미지마다 txt	class, x_center, y_center, w, h (0~1 정규화)

2. yml 파일 준비

- projects
 - my_car_detect_proj.yml

```

In [36]: import os
import json
import cv2
import matplotlib.pyplot as plt

def show_single_coco_image(
    coco_json_path,
    image_dir,
    image_index=0 # 시각화할 이미지 순서 (0부터 시작)
):
    # Load annotation JSON
    with open(coco_json_path, 'r') as f:
        coco = json.load(f)

    # Map image_id to image metadata
    image_info = coco['images'][image_index]
    image_id = image_info['id']
    file_name = image_info['file_name']
    img_path = os.path.join(image_dir, file_name)

    # Load image
    img = cv2.imread(img_path)
    if img is None:
        print(f"이미지를 불러올 수 없습니다: {img_path}")
        return

    # Index annotations for this image
    anns = [ann for ann in coco['annotations'] if ann['image_id'] == image_id]
    cat_id_to_name = {cat['id']: cat['name'] for cat in coco['categories']}

    # Draw bounding boxes
    for ann in anns:
        x, y, w, h = map(int, ann['bbox'])
        category_id = ann['category_id']
        label = cat_id_to_name.get(category_id, str(category_id))

        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(img, label, (x, y - 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)

    # Convert BGR to RGB for matplotlib
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Show image
    plt.figure(figsize=(10, 10))
    plt.imshow(img_rgb)
    plt.title(file_name)
    plt.axis('off')
    plt.show()

```

COCO JSON으로 이미지 한 장에 정답 박스를 그리는 함수다. COCO JSON은 `images` (이미지 id·파일명), `annotations` (박스마다 `image_id`, `category_id`, `bbox` [`x`, `y`, `w`, `h`]), `categories` (`id` ↔ 이름) 세 부분으로 되어 있다. `image_index` 번째 이미지의 `id`로 `annotations` 를 골라내고, `bbox` 는 왼쪽 위 + 너비·높이라서 오른쪽 아래를 (`x + w`, `y + h`) 로 계산해 그린다. 마지막으로 BGR → RGB로 바꿔 matplotlib으로 띄운다.

```
In [37]: # 🐞 코랩용 (원본)
# !ls -l
# 📖 노트북용: Windows 명령창에는 ls가 없다 → 파이썬으로 폴더 내용을 본다
for name in sorted(os.listdir('.')):
    print(name)
```

```
.git
.gitignore
LICENSE
__pycache__
backbone.py
benchmark
coco_eval.py
datasets
efficientdet
efficientdet_test.py
efficientdet_test_videos.py
efficientnet
output_soccer.mp4
output_soccer_frames
projects
readme.md
res
soccer.gif
test
train.py
tutorial
utils
weights
```

```
In [38]: # 🐞 코랩용 (원본)
# !pwd
# 📖 노트북용: Windows 명령창에는 pwd가 없다 → 파이썬으로 현재 폴더를 본다
os.getcwd()
```

```
Out[38]: 'C:\\Users\\L0Q5060\\Desktop\\cv-4\\Yet-Another-EfficientDet-Pytorch'
```

```
In [39]: # 🐞 코랩용 (원본)
# !unzip ./datasets/archive.zip -d ./datasets/car_detect
# 📖 노트북용: Windows 명령창에는 unzip이 없다 → 파이썬 zipfile로 같은 폴더에 푼다
import zipfile
with zipfile.ZipFile('./datasets/archive.zip') as z:
    z.extractall('./datasets/car_detect')
print(os.listdir('./datasets/car_detect'))
```

```
['Apply_Grayscale', 'No_Apply_Grayscale', 'SampleVideo_LowQuality.mp4', 'Sample_Video_HighQuality.mp4']
```

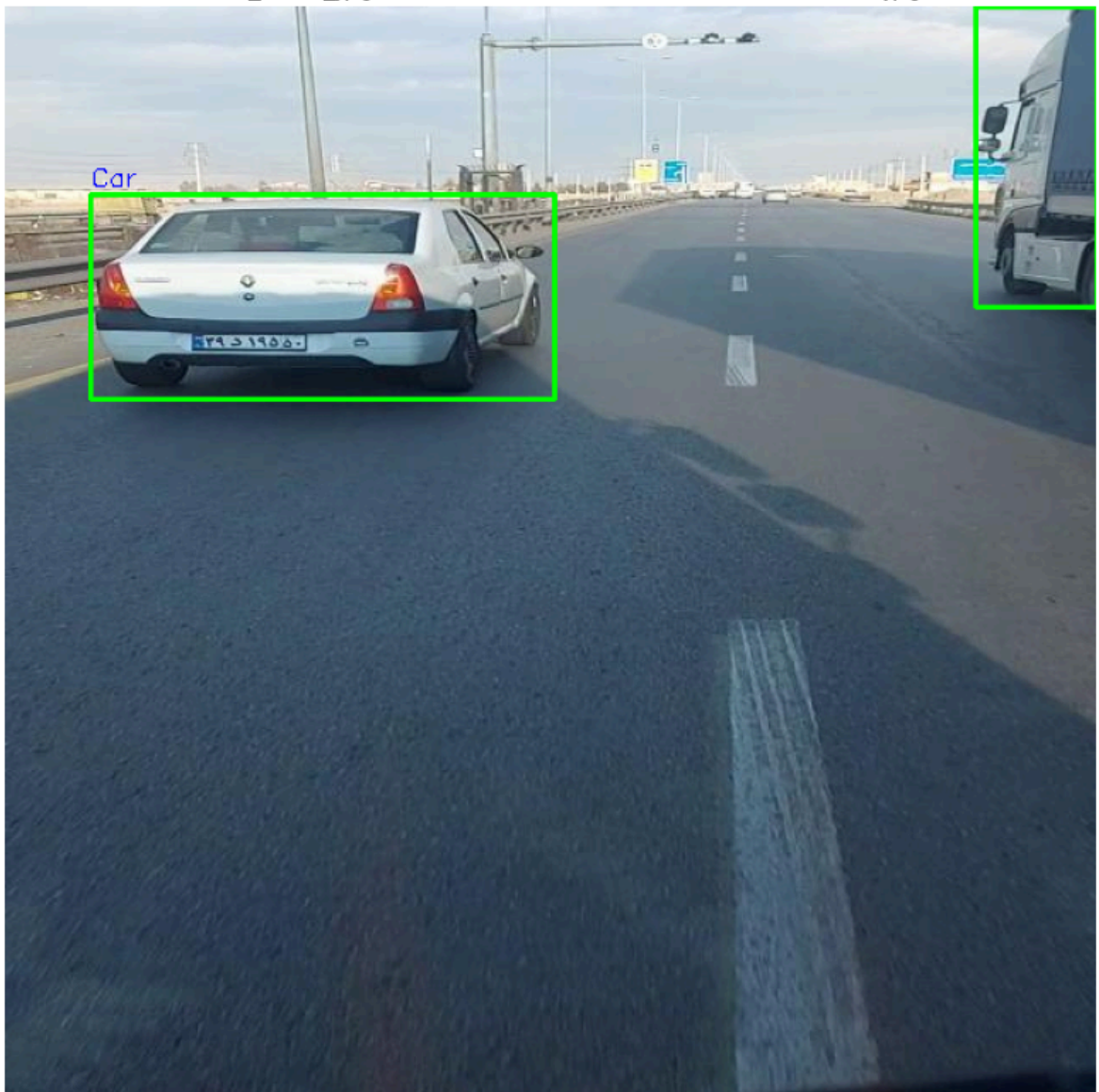
압축을 `datasets/car_detect` 에 푼다. 흑백,컬러 두 버전마다 COCO, YOLOv8, YOLOv9, TensorFlow 형식이 함께 들어 있고, 샘플 영상 2개도 있다.

내 컴퓨터에서 돌리려고 바꾼 곳

- `!pwd` → `os.getcwd()` (Windows 명령창에는 `pwd`가 없다).
 - `!ls -l` → `os.listdir` (Windows 명령창에는 `ls`가 없다).
 - `!unzip` → 파이썬 `zipfile` (Windows 명령창에는 `unzip`이 없다).
- 압축 파일과 푸는 위치는 원본과 같다.

```
In [40]: show_single_coco_image(  
        coco_json_path='./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/  
        Vehicles_Detection.v8i.coco/test/_annotations.coco.json',  
        image_dir='./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Vehic  
        les_Detection.v8i.coco/test',  
        image_index=0 # 첫 번째 이미지 시각화  
    )
```

frame_9486_jpg.rf.2280c498ddc7234aff225ca73be23450.jpg



컬러 버전 test 첫 장에 정답 박스를 그렸다. 고가도로 위에서 내려다본 장면이라 차가 작고 세로로 길게 찍혀 있다.

mean, std, anchor 계산

```
In [41]: import os
import cv2
import numpy as np
from glob import glob
from tqdm import tqdm

def compute_mean_std(image_dir, max_images=None):
    image_paths = sorted(glob(os.path.join(image_dir, '*.jpg')))
    if max_images:
        image_paths = image_paths[:max_images]

    means = []
    stds = []

    for path in tqdm(image_paths, desc="Computing mean/std"):
        img = cv2.imread(path) # BGR
        if img is None:
            continue
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = img / 255.0
        means.append(np.mean(img, axis=(0, 1)))
        stds.append(np.std(img, axis=(0, 1)))

    mean = np.mean(means, axis=0)
    std = np.mean(stds, axis=0)

    print(f"✅ RGB mean: {mean.tolist()}")
    print(f"✅ RGB std : {std.tolist()}")
    return mean.tolist(), std.tolist()
```

```
In [42]: compute_mean_std('./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Vehicles_Detection.v8i.coco/train', max_images=500) # 샘플 500장 기준
```

```
✅ RGB mean: [0.4639068720376132, 0.4723598790503507, 0.4700067273366889]
✅ RGB std : [0.16533415666407805, 0.16423948194385204, 0.16310458962308103]
```

```
Out[42]: ([0.4639068720376132, 0.4723598790503507, 0.4700067273366889],
 [0.16533415666407805, 0.16423948194385204, 0.16310458962308103])
```

정규화에 쓸 RGB 채널별 평균·표준편차를 데이터셋에서 직접 계산한다. 기본값 (ImageNet)은 (0.485, 0.456, 0.406) 인데, 데이터 색감이 많이 다르면 직접 구한 값을 쓴다. 이미지마다 세로·가로 방향으로 평균을 내 채널별 값 3개를 얻고 (axis=(0, 1)), 모든 이미지의 값을 다시 평균한다. max_images=500 이지만 train 이미지가 102장이라 102장으로 계산했다. 결과 평균은 R·G·B가 모두 0.46~0.47로 거의 같다. 색이 거의 없는 회색조 영상이라는 뜻이다.

```

In [43]: import json
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

def analyze_bbox_ratios_and_scales(coco_json_path, show_plots=True):

    with open(coco_json_path, 'r') as f:
        coco = json.load(f)

    # 바운딩 박스 정보 추출
    widths, heights, areas = [], [], []
    category_stats = defaultdict(list) # 카테고리별 통계

    # 카테고리 이름 매핑
    cat_id_to_name = {cat['id']: cat['name'] for cat in coco['categories']}

    for ann in coco['annotations']:
        x, y, w, h = ann['bbox']
        if w > 0 and h > 0: # 유효한 바운딩 박스만
            widths.append(w)
            heights.append(h)
            areas.append(w * h)

        # 카테고리별 분석을 위해
        cat_name = cat_id_to_name.get(ann['category_id'], 'unknown')
        category_stats[cat_name].append((w, h))

    widths = np.array(widths)
    heights = np.array(heights)
    areas = np.array(areas)

    # 1. Aspect Ratios 계산
    ratios = widths / heights

    # 2. Scales 계산 (면적의 제곱근)
    scales = np.sqrt(areas)

    # 3. 시각화
    if show_plots:
        plt.figure(figsize=(15, 5))

        # Aspect Ratios 히스토그램
        plt.subplot(1, 3, 1)
        plt.hist(ratios, bins=50, alpha=0.7, edgecolor='black')
        plt.axvline(np.median(ratios), color='red', linestyle='--',
                    label=f'Median: {np.median(ratios):.2f}')
        plt.xlabel('Aspect Ratio (W/H)')
        plt.ylabel('Frequency')
        plt.title('Distribution of Aspect Ratios')
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Scales 히스토그램
        plt.subplot(1, 3, 2)
        plt.hist(scales, bins=50, alpha=0.7, edgecolor='black')
        plt.axvline(np.median(scales), color='red', linestyle='--',
                    label=f'Median: {np.median(scales):.1f}')
        plt.xlabel('Scale (sqrt of area)')

```

```

plt.ylabel('Frequency')
plt.title('Distribution of Scales')
plt.legend()
plt.grid(True, alpha=0.3)

# Log scale로 스케일 분포 확인
plt.subplot(1, 3, 3)
plt.hist(np.log(scales), bins=50, alpha=0.7, edgecolor='black')
plt.xlabel('Log Scale')
plt.ylabel('Frequency')
plt.title('Distribution of Log Scales')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 4. 통계 정보 출력
print("📊 데이터셋 통계")
print(f"총 annotation 수: {len(widths)}")
print(f"평균 크기: {np.mean(scales):.1f}")
print(f"중앙값 크기: {np.median(scales):.1f}")
print(f"평균 비율: {np.mean(ratios):.2f}")
print(f"중앙값 비율: {np.median(ratios):.2f}")
print()

# 5. Anchor Ratios 추천 (20%, 50%, 80% percentile)
ratio_percentiles = np.percentile(ratios, [20, 50, 80])
print("🎯 추천 Anchor Ratios")
print("Percentile 기반 (20%, 50%, 80%):")

# 단일 값 형태 (EfficientDet/YOLO 스타일)
ratios_simple = [round(r, 2) for r in ratio_percentiles]
print(f"Simple format: {ratios_simple}")

# Tuple 형태 (일부 구현체용)
ratios_tuple = [(1.0, round(1/r, 2)) if r > 1 else (round(r, 2), 1.0)
                 for r in ratio_percentiles]
print(f"Tuple format: {ratios_tuple}")
print()

# 6. Anchor Scales 추천
scale_percentiles = np.percentile(scales, [20, 50, 80])
base_scale = scale_percentiles[1] # 중앙값을 기준으로
scales_normalized = [round(s / base_scale, 2) for s in scale_percentiles]

print("🔧 추천 Anchor Scales")
print(f"절대값: {[round(s, 1) for s in scale_percentiles]}")
print(f"정규화 (중앙값 기준): {scales_normalized}")
print(f"기준 크기: {base_scale:.1f}")
print()

# 7. 실제 사용 예시
print("💡 실제 설정 예시")
print("# EfficientDet/RetinaNet 스타일")
print(f"anchor_ratios = {ratios_simple}")
print(f"anchor_scales = {scales_normalized}")
print("")
print("# 또는 전통적인 설정")
print("anchor_ratios = [0.5, 1.0, 2.0]")

```

```

print("anchor_scales = [1.0, 1.26, 1.587] # 2^(0/3), 2^(1/3), 2^(2/3)")
print()

# 8. 카테고리별 분석 (상위 5개)
print("📄 주요 카테고리별 비율 분석")
cat_ratios = {}
for cat_name, boxes in category_stats.items():
    if len(boxes) >= 10: # 충분한 샘플이 있는 카테고리만
        boxes = np.array(boxes)
        cat_ratios[cat_name] = np.median(boxes[:, 0] / boxes[:, 1])

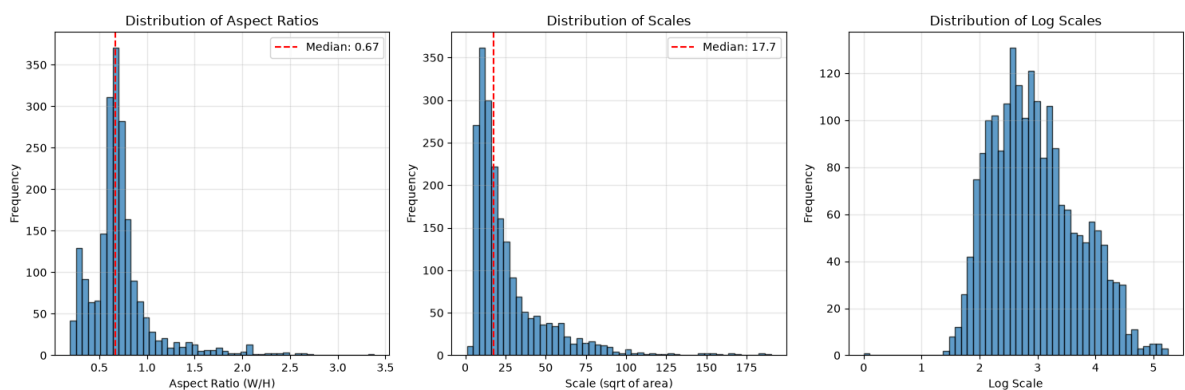
# 빈도 기준 정렬
sorted_cats = sorted(cat_ratios.items(),
                      key=lambda x: len(category_stats[x[0]]),
                      reverse=True)[:5]

for cat_name, median_ratio in sorted_cats:
    count = len(category_stats[cat_name])
    print(f" {cat_name}: {median_ratio:.2f} (samples: {count})")

return {
    'ratios_simple': ratios_simple,
    'ratios_tuple': ratios_tuple,
    'scales_normalized': scales_normalized,
    'scales_absolute': [round(s, 1) for s in scale_percentiles],
    'base_scale': base_scale,
    'stats': {
        'total_annotations': len(widths),
        'mean_ratio': np.mean(ratios),
        'median_ratio': np.median(ratios),
        'mean_scale': np.mean(scales),
        'median_scale': np.median(scales)
    }
}

```

In [44]: `analyze_bbox_ratios_and_scales('./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Vehicles_Detection.v8i.coco/train/_annotations.coco.json')`



📊 데이터셋 통계
총 annotation 수: 2069
평균 크기: 25.8
중앙값 크기: 17.7
평균 비율: 0.71
중앙값 비율: 0.67

🎯 추천 Anchor Ratios
Percentile 기반 (20%, 50%, 80%):
Simple format: [0.52, 0.67, 0.82]
Tuple format: [(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)]

🔧 추천 Anchor Scales
절대값: [9.6, 17.7, 37.6]
정규화 (중앙값 기준): [0.54, 1.0, 2.13]
... (2줄 생략) ...

💡 실제 설정 예시

```
# EfficientDet/RetinaNet 스타일
anchor_ratios = [0.52, 0.67, 0.82]
anchor_scales = [0.54, 1.0, 2.13]

# 또는 전통적인 설정
anchor_ratios = [0.5, 1.0, 2.0]
anchor_scales = [1.0, 1.26, 1.587] # 2^(0/3), 2^(1/3), 2^(2/3)
```

📋 주요 카테고리별 비율 분석
Car: 0.71 (samples: 1527)
Motorcycle: 0.31 (samples: 281)
Pickup: 0.56 (samples: 190)
Truck: 1.45 (samples: 47)
Bus: 0.36 (samples: 24)

```
Out[44]: {'ratios_simple': [0.52, 0.67, 0.82],
          'ratios_tuple': [(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)],
          'scales_normalized': [0.54, 1.0, 2.13],
          'scales_absolute': [9.6, 17.7, 37.6],
          'base_scale': 17.663521732655695,
          'stats': {'total_annotations': 2069,
                    'mean_ratio': 0.7142181132304681,
                    'median_ratio': 0.6666666666666666,
                    'mean_scale': 25.800453307673692,
                    'median_scale': 17.663521732655695}}
```

정답 박스들의 모양을 분석해 이 데이터에 맞는 앵커를 추천받는다. 앵커가 실제 물체 모양과 비슷할수록 모델이 조금만 고치면 되니 학습이 쉽다. 박스마다 비율(w / h)과 크기($\sqrt{w \times h}$)를 구해 분포를 그리고, 20-50-80% 지점 값으로 앵커 비율 3개를, 크기의 20-50-80% 지점을 중앙값으로 나눈 값으로 앵커 크기 3개를 추천한다.

결과를 보면 박스 2069개의 중앙 비율이 0.67로 **세로로 긴 박스**가 대부분이고, 중앙 크기가 17.7픽셀로 **작은 물체**가 많다. 추천값은 비율 $[(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)]$, 크기 $[0.54, 1.0, 2.13]$ 이고, 이 값을 yml에 넣는다.

```
In [45]: import json

def extract_obj_list(coco_json_path):
    with open(coco_json_path, 'r') as f:
        coco = json.load(f)

        obj_list = [cat['name'] for cat in sorted(coco['categories'], key=lambda x:
x['id'])]
        print("obj_list:", obj_list)
        return obj_list
```

```
In [46]: extract_obj_list('./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Ve
hicles_Detection.v8i.coco/train/_annotations.coco.json')
```

```
obj_list: ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']
```

```
Out[46]: ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']
```

`categories` 를 id 순서로 정렬해 이름만 뽑는다. 결과는 `['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']` 이다. 맨 앞의 `cars` (id 0)는 Roboflow가 자동으로 넣는 상위 카테고리이고, 실제 박스에는 `Bus ~ Truck` (id 1~5)만 쓰인다(코드 리뷰 3-4).

🔍 코드 리뷰 3-4 · `obj_list` 에 상위 카테고리 `cars` 가 들어감

- **문제:** 상위 카테고리 `cars` (id 0)까지 넣었다. 레포는 라벨을 `category_id - 1` 로 만들어 `Bus`(id 1)가 라벨 0이 된다
- **영향:** 모델이 "0번(`Bus`)"이라고 답하면 `cars` 로, "1번(`Car`)"이면 `Bus` 로 찍혀 이름이 한 칸씩 밀린다. 쓰지 않는 클래스 하나가 늘어난다
- **개선안:** `obj_list = ['Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']`
(아래 근거 1)

```
obj_list = [cat['name'] for cat in sorted(coco['categories'], key=lambda
x: x['id'])
            if cat['supercategory'] != 'none'] # ['Bus', 'Car', 'Motor
cycle', 'Pickup', 'Truck']
```

- Yet-Another-EfficientDet-Pytorch
 - projects
 - my_car_detect_proj.yml
 - datasets
 - car_detect
 - my_car_detect
 - annotations
 - instances_train.json
 - instances_valid.json
 - instances_test.json
 - train
 - XXX.jpg
 - XXX.jpg
 - valid
 - XXX.jpg
 - XXX.jpg
 - test
 - XXX.jpg
 - XXX.jpg

```

In [47]: import os
import shutil
import json
from pathlib import Path

# 1) 설정: 원본/대상 경로 및 split 정의
SRC_ROOT = Path("datasets/car_detect/Apply_Grayscale/Apply_Grayscale/Vehicles_De
tection.v9i.coco")
DST_ROOT = Path("datasets/my_car_detect_proj")
ANNOT_DIR = DST_ROOT / "annotations"
SPLITS = ["train", "valid", "test"]

# 2) 대상 디렉토리 생성
for p in [DST_ROOT, ANNOT_DIR] + [DST_ROOT / s for s in SPLITS]:
    p.mkdir(parents=True, exist_ok=True)

# 3) split별 이미지 복사 및 annotation 변환·저장
for split in SPLITS:
    src_split = SRC_ROOT / split
    dst_split = DST_ROOT / split
    dst_split.mkdir(exist_ok=True)

    # 3-1) 이미지 파일(.jpg/.png) 복사
    for ext in ("*.jpg", "*.png"):
        for img_path in src_split.glob(ext):
            shutil.copy(img_path, dst_split / img_path.name)

    # 3-2) annotation JSON 변환
    # - 원본 JSON 불러오기
    # - COCO to EfficientDet-PyTorch 형식 변환
    # - instances_{split}.json으로 저장
    for json_path in src_split.glob("*.json"):
        with open(json_path, "r", encoding="utf-8") as f:
            before = json.load(f)

    # 새 구조 준비
    tobe = {
        "images": [],
        "type": "instances",
        "annotations": [],
        "categories": []
    }

    # images 필드
    for img in before.get("images", []):
        tobe["images"].append({
            "file_name": img["file_name"],
            "height": img["height"],
            "width": img["width"],
            "id": img["id"]
        })

    # annotations 필드
    for ann in before.get("annotations", []):
        tobe["annotations"].append({
            "id": ann["id"],
            "image_id": ann["image_id"],
            "category_id": ann["category_id"],
            "bbox": ann["bbox"],

```

```

        "area": ann["area"],
        "iscrowd": ann.get("iscrowd", 0),
        "ignore": 0,
        "segmentation": ann.get("segmentation", [])
    })

# categories 폴더
for cat in before.get("categories", []):
    tobe["categories"].append({
        "supercategory": cat.get("supercategory", ""),
        "id": cat["id"],
        "name": cat["name"]
    })

# 저장
dst_json = ANNOT_DIR / f"instances_{split}.json"
with open(dst_json, "w", encoding="utf-8") as f:
    json.dump(tobe, f, ensure_ascii=False, indent=2)

print(f"[{split}] {json_path.name} → {dst_json.name}")

print("✅ 전체 폴더 구조 및 annotation 변환 완료!")

```

[train] _annotations.coco.json → instances_train.json

[valid] _annotations.coco.json → instances_valid.json

[test] _annotations.coco.json → instances_test.json

✅ 전체 폴더 구조 및 annotation 변환 완료!

흑백 버전(v9i)을 레포 구조로 옮긴다. split(train/valid/test)마다 이미지를 복사하고, JSON에서 필요한 키만 골라(type , ignore 추가) annotations/instances_{split}.json으로 저장한다. Path 는 / 연산자로 경로를 이어 붙일 수 있고, mkdir(parents=True, exist_ok=True) 는 중간 폴더까지 만들고 이미 있으면 넘어간다.

🔍 코드 리뷰 3-1 · 분석·학습·예측 데이터 버전이 다름

- **문제:** 확인·mean/std·앵커·클래스 분석은 컬러(No_Apply, v8i), 학습은 흑백(Apply_Grayscale, v9i), 마지막 예측은 다시 컬러 test 이미지로 한다
- **영향:** 학습 데이터와 다른 데이터로 정규화 값과 앵커를 정했고, 학습 때 본 적 없는 컬러 이미지로 결과를 판단한다
- **개선안:** 경로를 변수 하나(DATA_ROOT)로 정하고 분석·학습·예측에 같은 버전을 쓴다

```
DATA_ROOT = Path('datasets/car_detect/Apply_Grayscale/Apply_Grayscale/Vehicles_Detection.v9i.coco')
```

```
compute_mean_std(str(DATA_ROOT / 'train'))
```

```
analyze_bbox_ratios_and_scales(str(DATA_ROOT / 'train' / '_annotations.coco.json'))
```

```
# 예측도 DATA_ROOT / 'test' 의 이미지로
```

```

In [48]: import os
import shutil
from pathlib import Path

# 원본 루트 (변경 필요 시 수정)
SRC_ROOT = Path("datasets/car_detect/Apply_Grayscale/Apply_Grayscale/Vehicles_De
tection.v9i.coco")

# 대상 루트
DST_ROOT = Path("datasets/my_car_detect_proj")
ANNOT_DIR = DST_ROOT / "annotations"

# splits 정보
SPLITS = ["train", "valid", "test"]

# annotation 파일 패턴 (원본 디렉토리 안에서 *.json 찾기)
ANNOT_PATTERN = "_annotations.coco.json" # test에만 있다면 'test'만 복사됩니다.

# 1) 대상 디렉토리 생성
for d in [DST_ROOT, ANNOT_DIR] + [DST_ROOT / s for s in SPLITS]:
    d.mkdir(parents=True, exist_ok=True)

# 2) 이미지 파일 복사 및 annotation 파일 복사
for split in SPLITS:
    src_split = SRC_ROOT / split
    dst_split = DST_ROOT / split

    # 이미지(.jpg/.png) 복사
    for ext in ("*.jpg", "*.png"):
        for img_path in src_split.glob(ext):
            shutil.copy(img_path, dst_split / img_path.name)

    # annotation JSON 복사 (split별로 이름 바꿔 복사)
    # 만약 train/valid 폴더에도 JSON이 있다면 동일 로직으로 처리됩니다.
    for json_path in src_split.glob("*.json"):
        dst_json = ANNOT_DIR / f"instances_{split}.json"
        shutil.copy(json_path, dst_json)
        print(f"Copied annotation for {split}: {json_path.name} → {dst_json.nam
e}")

print("✅ 폴더 구조 변환 완료!")

```

Copied annotation for train: _annotations.coco.json → instances_train.json

Copied annotation for valid: _annotations.coco.json → instances_valid.json

Copied annotation for test: _annotations.coco.json → instances_test.json

✅ 폴더 구조 변환 완료!

같은 일을 하는 두 번째 셀로, 이번에는 JSON을 손대지 않고 이름만 바꿔 복사한다. 같은 파일을 덮어쓰므로 최종으로 남는 것은 이 셀의 결과(원본 JSON 그대로)다(코드 리뷰 3-2).

🔍 코드 리뷰 3-2 · 같은 일을 하는 두 셀

- **문제:** 두 셀이 같은 `instances_*.json` 을 만들고, 두 번째가 첫 번째 결과를 덮어 쓴다
- **영향:** 첫 번째 셀의 변환 작업이 의미가 없어지고, 어느 쪽이 쓰였는지 헷갈린다
- **개선안:** 둘 중 하나만 남긴다 (레포는 원본 COCO JSON도 그대로 읽는다)

```
project_name: car_detect_proj  # also the folder name of the dataset tha
t under data_path folder
train_set: train
val_set: valid
num_gpus: 0

# mean and std in RGB order, actually this part should remain unchanged
as long as your dataset is similar to coco.
mean: [0.4639068720376132, 0.4723598790503507, 0.4700067273366889]
std: [0.16533415666407805, 0.16423948194385204, 0.16310458962308103]

# this is coco anchors, change it if necessary
anchors_scales: '[0.54, 1.0, 2.13]'
anchors_ratios: '[(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)]'

# must match your dataset's category_id.
# category_id is one_indexed,
# for example, index of 'car' here is 2, while category_id of is 3
obj_list: ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']
```

```
In [49]: # 📖 [추가] 노트북용 셀 (원본에 없음): 원본은 위 yml 내용을 마크다운으로만 보여
주고 파일은 직접 만들었다 → 파이썬으로 같은 파일을 만든다
# 📖 코랩용 (원본 메모): project_name: car_detect_proj / num_gpus: 0
# 📖 노트북용: project_name은 데이터 폴더 이름(my_car_detect_proj)과 같아야 데이
터를 찾고, num_gpus가 0이면 train.py가 CPU로 학습해서 1로 바꿨다
# 📖 train.py는 yml을 Windows 기본 인코딩(cp949)으로 읽어서, 파일 안에는 한글·이
모지 주석을 넣지 않는다 (원본의 영어 주석만 둔다)
yaml = """project_name: my_car_detect_proj # also the folder name of the dataset
that under data_path folder
train_set: train
val_set: valid
num_gpus: 1

# mean and std in RGB order, actually this part should remain unchanged as long
as your dataset is similar to coco.
mean: [0.4639068720376132, 0.4723598790503507, 0.4700067273366889]
std: [0.16533415666407805, 0.16423948194385204, 0.16310458962308103]

# this is coco anchors, change it if necessary
anchors_scales: '[0.54, 1.0, 2.13]'
anchors_ratios: '[(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)]'

# must match your dataset's category_id.
# category_id is one_indexed,
# for example, index of 'car' here is 2, while category_id of is 3
obj_list: ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']
"""

with open('projects/my_car_detect_proj.yaml', 'w', encoding='ascii') as f:
    f.write(yaml)
print(open('projects/my_car_detect_proj.yaml').read())
```

```
project_name: my_car_detect_proj # also the folder name of the dataset that under
data_path folder
train_set: train
val_set: valid
num_gpus: 1

# mean and std in RGB order, actually this part should remain unchanged as long as
your dataset is similar to coco.
mean: [0.4639068720376132, 0.4723598790503507, 0.4700067273366889]
std: [0.16533415666407805, 0.16423948194385204, 0.16310458962308103]

# this is coco anchors, change it if necessary
anchors_scales: '[0.54, 1.0, 2.13]'
anchors_ratios: '[(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)]'

# must match your dataset's category_id.
# category_id is one_indexed,
# for example, index of 'car' here is 2, while category_id of is 3
obj_list: ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']
```

train.py 가 읽는 프로젝트 설정 파일이다. project_name 은 데이터 폴더 이름(datasets/{project_name}/)이자 학습 결과 폴더 이름(logs/{project_name}/)이다. mean · std 는 위에서 계산한 값, anchors_* 는 추천받은 값이고, 앵커는 문자열로 적어 두면 train.py 가 eval() 로 리스트로 바꾼다. obj_list 의 길이가 곧 클래스 수(6)가 된다.

 **내 컴퓨터에서 돌리려고 바꾼 곳** — 원본에 없는 셀이다. 원본은 yaml 내용을 마크다운으로만 보여 주고 파일은 직접 만들었다. project_name 을 데이터 폴더 이름에 맞추고 num_gpus 를 1로 바꿨으며, train.py 가 yaml을 cp949로 읽어서 파일 안에는 한글 주석을 넣지 않았다.

🔍 코드 리뷰 3-3 · yaml 메모가 실제와 다름

- **문제:** project_name: car_detect_proj 는 폴더 이름(my_car_detect_proj)과 다르고, num_gpus: 0 이면 CPU로 학습한다
- **영향:** 메모대로 만들면 데이터를 못 찾고, GPU가 있어도 쓰지 않는다
- **개선안:** 폴더 이름과 맞추고 num_gpus: 1

```
project_name: my_car_detect_proj    # datasets/ 아래 폴더 이름과 같게
num_gpus: 1
```

학습

- scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3, verbose=True)
- 에러 발생 시 => 아래와 같이 수정
- scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3)

```
In [50]: #  [추가] 노트북용 셀 (원본에 없음): 원본은 위 안내대로 train.py를 직접 열어
고치라고만 했다 → 코드로 같은 수정을 한다
src = open('train.py', encoding='utf-8').read()
src = src.replace('ReduceLROnPlateau(optimizer, patience=3, verbose=True)', 'Red
uceLROnPlateau(optimizer, patience=3)')
open('train.py', 'w', encoding='utf-8').write(src)
print([line.strip() for line in src.splitlines() if 'ReduceLROnPlateau' in lin
e])
```

```
['scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=3)']
```

ReduceLROnPlateau 는 검증 손실이 3번 연속 좋아지지 않으면 학습률을 줄이는 스케줄러다. 최신 PyTorch에서 verbose 인자가 없어서 에러가 나므로 원본 안내대로 지웠다.

 **내 컴퓨터에서 돌리려고 바꾼 곳** — 원본에 없는 셀이다. 원본 안내대로 train.py 를 손으로 고치는 대신 코드로 고쳤다.

```
In [51]: # 🐞 코랩용 (원본)
# !pip install tensorboardX
# 📖 노트북용: 이 PC에서 !pip는 시스템 파이썬에 설치한다 → %pip는 지금 커널(.venv)에 설치한다
%pip install tensorboardX
```

Requirement already satisfied: tensorboardX in C:\Users\LOQ5060\Desktop\cv-4\.venv\Lib\site-packages (2.6.5)
Requirement already satisfied: numpy in C:\Users\LOQ5060\Desktop\cv-4\.venv\Lib\site-packages (from tensorboardX) (1.26.4)
Requirement already satisfied: packaging in C:\Users\LOQ5060\Desktop\cv-4\.venv\Lib\site-packages (from tensorboardX) (26.3)
Requirement already satisfied: protobuf<=3.20 in C:\Users\LOQ5060\Desktop\cv-4\.venv\Lib\site-packages (from tensorboardX) (7.36.2)
Note: you may need to restart the kernel to use updated packages.

```
In [52]: # 🐞 코랩용 (원본)
# !pwd
# 📖 노트북용: Windows 명령창에는 pwd가 없다 → 파이썬으로 현재 폴더를 본다
os.getcwd()
```

Out[52]: 'C:\\Users\\LOQ5060\\Desktop\\cv-4\\Yet-Another-EfficientDet-Pytorch'

train.py가 학습 기록에 쓰는 tensorboardX를 설치한다. 이 커널에는 이미 설치되어 있다.

📖 내 컴퓨터에서 돌리려고 바꾼 곳

- !pwd → os.getcwd() (Windows 명령창에는 pwd가 없다).
- !pip → %pip (이 PC에서 !pip는 시스템 파이썬에 설치한다).

```
In [53]: # 🐞 코랩용 (원본)
# ! python train.py -c 0 -p my_car_detect_proj --head_only True --lr 1e-3 --batch_size 16 --load_weights weights/efficientdet-d0.pth --num_epochs 10 --save_interval 100
# 📖 노트북용: 이 PC에서 python은 시스템 파이썬이라 pycocotools.tensorboardX가 없다 → 지금 커널(.venv)의 파이썬(sys.executable)으로 실행
# 끝에 -n 0 추가: 워커 기본값 12는 Windows에서 에폭마다 프로세스 12개를 새로 띄워 에폭당 3분 넘게 걸린다 → 0이면 약 15초
import sys
!{sys.executable} train.py -c 0 -p my_car_detect_proj --head_only True --lr 1e-3 --batch_size 16 --load_weights weights/efficientdet-d0.pth --num_epochs 10 --save_interval 100 -n 0
```

```
loading annotations into memory...
Done (t=0.02s)
creating index...
index created!
loading annotations into memory...
Done (t=0.01s)
creating index...
index created!
[Warning] Ignoring Error(s) in loading state_dict for EfficientDetBackbone:
    size mismatch for classifier.header.pointwise_conv.conv.weight: copying a
param with shape torch.Size([810, 64, 1, 1]) from checkpoint, the shape in current
model is torch.Size([54, 64, 1, 1]).
    size mismatch for classifier.header.pointwise_conv.conv.bias: copying a pa
ram with shape torch.Size([810]) from checkpoint, the shape in current model is to
rch.Size([54]).
[Warning] Don't panic if you see this, this might be because you load a pretrained
weights with different number of classes. The rest of the weights should be loaded
already.
[Info] loaded weights: efficientdet-d0.pth, resuming checkpoint from step: 0
[Info] freezed backbone
Val. Epoch: 0/10. Classification loss: 13426.66016. Regression loss: 3.21334. Total loss: 13429.87350
Val. Epoch: 1/10. Classification loss: 8466.17578. Regression loss: 2.86537. Total loss: 8469.04115
Val. Epoch: 2/10. Classification loss: 4897.40967. Regression loss: 2.58938. Total loss: 4899.99904
Val. Epoch: 3/10. Classification loss: 2570.54639. Regression loss: 2.51814. Total loss: 2573.06453
Val. Epoch: 4/10. Classification loss: 1176.40808. Regression loss: 2.32303. Total loss: 1178.73111
Val. Epoch: 5/10. Classification loss: 506.79483. Regression loss: 2.44872. Total loss: 509.24355
Val. Epoch: 6/10. Classification loss: 270.17657. Regression loss: 2.28910. Total loss: 272.46568
Val. Epoch: 7/10. Classification loss: 151.06752. Regression loss: 2.33367. Total loss: 153.40119
Val. Epoch: 8/10. Classification loss: 99.94410. Regression loss: 2.17783. Total loss: 102.12193
Val. Epoch: 9/10. Classification loss: 82.39716. Regression loss: 2.16339. Total loss: 84.56055
```

1단계 학습 — head만 학습. COCO 가중치에서 출발해 백본(EfficientNet)·BiFPN은 고정하고, 마지막 박스·클래스 예측층(head)만 10에폭 학습한다.

인자	의미
<code>-c 0</code>	D0 모델
<code>-p my_car_detect_proj</code>	<code>projects/my_car_detect_proj.yml</code> 설정 사용
<code>--head_only True</code>	백본·BiFPN은 얼리고 head만 학습. 적은 데이터로 COCO에서 배운 특징 추출 능력은 그대로 쓰고 차 종류만 새로 배운다
<code>--lr 1e-3 / --batch_size 16</code>	학습률 / 한 번에 16장씩
<code>--load_weights</code>	COCO 가중치에서 시작. 클래스 수(90 → 6)가 달라 classifier 마지막 층은 모양이 안 맞아 건너뛰다(<code>size mismatch</code> 경고)
<code>--num_epochs 10 / --save_interval 100</code>	10에폭 / 100 step마다 저장(검증 손실이 좋아질 때도 저장)

검증 손실은 Classification이 13427 → 82로 크게 줄었고, Regression은 3.21 → 2.16으로 천천히 줄었다.

에폭	0	3	6	9
Classification loss	13426.7	2570.5	270.2	82.4
Regression loss	3.21	2.52	2.29	2.16

첫 손실이 수만 단위로 큰 건 클래스 head가 새로 시작해서이기도 하고, 이 레포의 Focal Loss가 "정답과 IoU 0.5 이상 겹치는 앵커 수"로 나누는데 작은 차는 앵커와 잘 안 겹쳐서 나누는 수가 작아지기 때문이기도 하다. 원본 Colab도 16233 → 108로 비슷하게 줄었다.

 **내 컴퓨터에서 돌리려고 바꾼 곳** — `python` → `sys.executable`
(이 PC의 `python` 은 `pycocotools` 가 없는 시스템 파이썬이다), 끝에 `-n 0` 추가(워커 12개는 Windows에서 에폭당 3분 넘게 걸려서 0으로 줄였고, 에폭당 약 15초가 됐다). 학습 인자는 원본과 같다.

코드 리뷰 3-8 · 워커 수 기본값 12

- **문제:** `num_workers` 기본값 12를 그대로 썼다
- **영향:** 원본 Colab 로그에도 "워커 12개는 너무 많다(권장 2개)" 경고가 있고, Windows에서는 에폭당 3분 넘게 걸렸다
- **개선안:** 환경에 맞게 `-n` 을 지정한다

`-n 2` # Colab 권장값 (Windows에서는 `-n 0`)

```
In [54]: # 🐞 코랩용 (원본)
# ! python train.py -c 0 -p my_car_detect_proj --head_only False --lr 1e-3 --batch_size 16 --load_weights './logs/my_car_detect_proj/efficientdet-d0_9_80.pth' --num_epochs 30 --save_interval 100
# 📖 노트북용: 커널(.venv) 파이썬으로 실행 + 끝에 -n 0 (이유는 1단계와 같다)
!{sys.executable} train.py -c 0 -p my_car_detect_proj --head_only False --lr 1e-3 --batch_size 16 --load_weights './logs/my_car_detect_proj/efficientdet-d0_9_80.pth' --num_epochs 30 --save_interval 100 -n 0
```

```
loading annotations into memory...
Done (t=0.01s)
creating index...
index created!
loading annotations into memory...
Done (t=0.00s)
creating index...
index created!
using weights logs//my_car_detect_proj\efficientdet-d0_9_80.pth
[Info] loaded weights: efficientdet-d0_9_80.pth, resuming checkpoint from step: 80
Val. Epoch: 10/30. Classification loss: 17.21909. Regression loss: 4.36739. Total loss: 21.58648
Val. Epoch: 11/30. Classification loss: 5.23777. Regression loss: 3.42106. Total loss: 8.65883
checkpoint...
Val. Epoch: 12/30. Classification loss: 3.70164. Regression loss: 3.19924. Total loss: 6.90088
Val. Epoch: 13/30. Classification loss: 2.67050. Regression loss: 3.05593. Total loss: 5.72643
... (4줄 생략) ...
Val. Epoch: 18/30. Classification loss: 1.08227. Regression loss: 2.43714. Total loss: 3.51941
Val. Epoch: 19/30. Classification loss: 1.05320. Regression loss: 2.39271. Total loss: 3.44591
Val. Epoch: 20/30. Classification loss: 1.03429. Regression loss: 2.37856. Total loss: 3.41285
Val. Epoch: 21/30. Classification loss: 1.01522. Regression loss: 2.33319. Total loss: 3.34841
Val. Epoch: 22/30. Classification loss: 0.98779. Regression loss: 2.52833. Total loss: 3.51612
Val. Epoch: 23/30. Classification loss: 0.92806. Regression loss: 2.29564. Total loss: 3.22370
checkpoint...
Val. Epoch: 24/30. Classification loss: 0.88444. Regression loss: 2.14318. Total loss: 3.02762
Val. Epoch: 25/30. Classification loss: 0.81710. Regression loss: 2.36063. Total loss: 3.17774
Val. Epoch: 26/30. Classification loss: 0.78581. Regression loss: 2.36174. Total loss: 3.14755
Val. Epoch: 27/30. Classification loss: 0.77805. Regression loss: 2.31786. Total loss: 3.09591
Val. Epoch: 28/30. Classification loss: 0.76313. Regression loss: 2.47246. Total loss: 3.23560
Val. Epoch: 29/30. Classification loss: 0.73926. Regression loss: 2.68141. Total loss: 3.42067
```

2단계 학습 — 전체 학습. `--head_only False` 로 백본까지 모두 학습하고, 1단계 마지막 가중치(`_9_80`: 9번째 에폭, 80번째 step)에서 이어서 시작한다.

로그의 에폭 번호가 10부터 시작한다(코드 리뷰 3-6). Classification은 17.2 → 0.74까지 계속 줄었지만, Regression은 4.37에서 24에폭 2.14까지 줄다가 이후 2.3~2.7 사이에서 오르내렸다. 검증 Total loss가 가장 낮았던 24에폭(3.03)이 마지막으로 저장됐다. 원본 Colab에서는 2단계 로그가 중간에 끊겨 검증 결과가 없었다.

 **내 컴퓨터에서 돌리려고 바꾼 곳** — 1단계와 같이 `sys.executable` 과 `-n 0` 만 바꿨다.

코드 리뷰 3-5 · 1단계 가중치 파일 이름을 손으로 적음

- **문제:** 1단계 가중치 파일 이름(`_9_80`)을 손으로 적었다
- **영향:** 데이터 수나 배치 크기가 바뀌면 이름이 달라져 에러가 난다
- **개선안:** 1단계 폴더에서 최신 파일을 찾아 넘긴다(`-w last` 도 가능)

```
!{sys.executable} train.py -c 0 -p my_car_detect_proj --head_only False
... --load_weights last
```

코드 리뷰 3-6 · `--num_epochs` 가 총 에폭 수

- **문제:** `--num_epochs 30` 은 1단계 10에폭을 포함한 수다
- **영향:** 로그가 에폭 10부터 시작하고, 실제로 더 학습하는 건 30이 아니라 **20에폭**이다
- **개선안:** 원하는 총 에폭 수로 적는다

```
--num_epochs 40 # 1단계 10에폭 + 2단계 30에폭
```

코드 리뷰 3-7 · 전체 학습도 같은 학습률

- **문제:** 전체 미세조정도 1단계와 같은 `1r 1e-3` 으로 한다
- **영향:** 이미 배운 백본 특징이 크게 흔들릴 수 있다
- **개선안:** 전체 학습은 `1e-4` 처럼 낮춘다

```
!{sys.executable} train.py ... --head_only False --lr 1e-4 ...
```

```
In [55]: %cd logs/my_car_detect_proj
# 🐞 코랩용 (원본)
# weight_file = !ls -Art | grep efficientdet
# 📖 노트북용: Windows 명령창에는 ls.grep이 없다 → dir /b(이름만) /od(오래된 것
#           부터 시간순)로 같은 목록을 받는다
weight_file = !dir /b /od efficientdet*
weight_file
```

C:\Users\LOQ5060\Desktop\cv-4\Yet-Another-EfficientDet-Pytorch\logs\my_car_detect_proj

```
Out[55]: ['efficientdet-d0_0_8.pth',
'efficientdet-d0_1_16.pth',
'efficientdet-d0_2_24.pth',
'efficientdet-d0_3_32.pth',
'efficientdet-d0_4_40.pth',
'efficientdet-d0_5_48.pth',
'efficientdet-d0_6_56.pth',
'efficientdet-d0_7_64.pth',
'efficientdet-d0_8_72.pth',
'efficientdet-d0_9_80.pth',
'efficientdet-d0_10_88.pth',
'efficientdet-d0_11_96.pth',
'efficientdet-d0_12_100.pth',
'efficientdet-d0_12_104.pth',
'efficientdet-d0_13_112.pth',
'efficientdet-d0_14_120.pth',
'efficientdet-d0_15_128.pth',
'efficientdet-d0_16_136.pth',
'efficientdet-d0_17_144.pth',
'efficientdet-d0_18_152.pth',
'efficientdet-d0_19_160.pth',
'efficientdet-d0_20_168.pth',
'efficientdet-d0_21_176.pth',
'efficientdet-d0_23_192.pth',
'efficientdet-d0_24_200.pth']
```

```
In [56]: %cd ..
%cd ..
```

```
C:\Users\LOQ5060\Desktop\cv-4\Yet-Another-EfficientDet-Pytorch\logs
C:\Users\LOQ5060\Desktop\cv-4\Yet-Another-EfficientDet-Pytorch
```

가중치 폴더로 이동해 원본은 `ls -Art` (수정 시간 오름차순)로 파일 목록을 받아 이름에 `efficientdet`이 있는 것만 리스트로 담는다. 노트북용 `dir /b /od efficientdet*` 도 같은 목록을 만든다. `weight_file[-1]` 이 가장 최근에 저장된 가중치다. 그다음 `%cd ..` 두 번으로 레포 폴더로 돌아온다. 이번 실행에서는 1단계 10개, 2단계 15개(100 step마다 저장된 `_12_100` 포함)가 저장됐고, 마지막 파일은 검증 손실이 가장 낮았던 `efficientdet-d0_24_200.pth` 다.

 내 컴퓨터에서 돌리려고 바꾼 곳 — `ls -Art | grep` → `dir /b /od`
`d` (Windows 명령창에는 `ls-grep`이 없다). `%cd` 는 원본 그대로다.

```
In [57]: # 🐞 코랩용 (원본)
# ! python coco_eval.py -c 0 -p my_car_detect_proj -w "logs/my_car_detect_proj/{weight_file[-1]}"
# 📓 노트북용: 커널(.venv) 파이썬으로 실행 (이유는 1단계와 같다)
!{sys.executable} coco_eval.py -c 0 -p my_car_detect_proj -w "logs/my_car_detect_proj/{weight_file[-1]}"
```

```

running coco-style evaluation on project my_car_detect_proj, weights logs/my_car_d
etect_proj/efficientdet-d0_24_200.pth...
loading annotations into memory...
Done (t=0.00s)
creating index...
index created!
Loading and preparing results...
DONE (t=0.66s)
creating index...
index created!
BBox
Running per image evaluation...
Evaluate annotation type *bbox*
DONE (t=1.00s).
Accumulating evaluation results...
DONE (t=0.11s).
Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.005
Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.011
Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.003
Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.009
Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.022
Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.003
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.001
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.029
Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.099
Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.027
Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.198
Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.331

```

coco_eval.py 가 yml의 val_set (valid) 이미지로 예측해 valid_bbox_results.json 을 만들고, pycocotools 의 COCOeval 로 정답과 비교해 12개 지표를 낸다.

지표	뜻
AP @[IoU=0.50:0.95]	대표 점수. IoU 기준을 0.5~0.95로 바꿔 가며 낸 평균 (YOLO의 mAP50-95)
AP @[IoU=0.50] / 0.75	박스가 50% / 75% 이상 겹치면 정답
AP small / medium / large	물체 크기별 AP (32 ² 미만 / 32 ² ~96 ² / 96 ² 이상 픽셀)
AR maxDets=1/10/100	이미지당 최대 1·10·100개만 볼 때의 재현율

결과는 AP@[.5:.95] **0.005**, AP50 **0.011**로 차를 거의 맞히지 못했다(원본 Colab은 1단계 가중치로 0.000). 크기별 재현율(AR@100)이 small 0.027, medium 0.198, large 0.331로, 작은 차일수록 못 찾는다. 앞의 분석에서 차 박스의 중앙 크기는 17.7픽셀이었는데, 이미지를 512로 줄이면 더 작아진다. D0의 가장 작은 앵커(P3, 4 × stride 8 × 0.54 ≈ 17픽셀)와 IoU 0.5 이상 겹치기 어려운 크기다. 학습 데이터도 136장으로 적다.

 내 컴퓨터에서 돌리려고 바꾼 곳 — python → sys.executable 만 바꿨다.

🔍 코드 리뷰 3-9 · valid로만 평가

- **문제:** coco_eval.py 는 yml의 val_set (valid)으로만 채점한다
- **영향:** 학습 중 가장 좋은 가중치를 고를 때도 valid를 썼으니, 성능이 실제보다 좋게 나올 수 있다
- **개선안:** test 셋으로도 채점한다

```
# projects/my_car_detect_proj_test.yml : 내용은 같고 val_set만 test로  
!{sys.executable} coco_eval.py -c 0 -p my_car_detect_proj_test -w "logs/  
my_car_detect_proj/{weight_file[-1]}"
```

```
In [58]: os.path.exists('./datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Vehi  
cles_Detection.v8i.coco/test/frame_0990_jpg.rf.a3223422e734a57442ee34a58d24d4b4.  
jpg')
```

```
Out[58]: True
```

```

In [59]: import torch
from torch.backends import cudnn

from backbone import EfficientDetBackbone
import cv2
import matplotlib.pyplot as plt
import numpy as np

from efficientdet.utils import BBoxTransform, ClipBoxes
from utils.utils import preprocess, invert_affine, postprocess

compound_coef = 0
force_input_size = None # set None to use default size
img_path = './datasets/car_detect/No_Apply_Grayscale/No_Apply_Grayscale/Vehicles
_Detection.v8i.coco/test/frame_0990_jpg.rf.a3223422e734a57442ee34a58d24d4b4.jpg'

threshold = 0.4
iou_threshold = 0.2

use_cuda = True
use_float16 = False

obj_list = ['cars', 'Bus', 'Car', 'Motorcycle', 'Pickup', 'Truck']

# tf bilinear interpolation is different from any other's, just make do
input_sizes = [512, 640, 768, 896, 1024, 1280, 1280, 1536]
input_size = input_sizes[compound_coef] if force_input_size is None else force_i
nput_size
ori_imgs, framed_imgs, framed metas = preprocess(img_path, max_size=input_size)

if use_cuda:
    x = torch.stack([torch.from_numpy(fi).cuda() for fi in framed_imgs], 0)
else:
    x = torch.stack([torch.from_numpy(fi) for fi in framed_imgs], 0)

x = x.to(torch.float32 if not use_float16 else torch.float16).permute(0, 3, 1,
2)

model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=len(obj_li
st),

                                # replace this part with your project's anchor conf
                                ig
                                ratios=[(1.0, 1.0), (1.3, 0.8), (1.9, 0.5)],
                                scales=[2 ** 0, 2 ** (1.0 / 3.0), 2 ** (2.0 / 3.
0)])

model.load_state_dict(torch.load('logs/my_car_detect_proj/'+weight_file[-1]))
model.requires_grad_(False)
model.eval()

if use_cuda:
    model = model.cuda()
if use_float16:
    model = model.half()

with torch.no_grad():
    features, regression, classification, anchors = model(x)

```

```

regressBoxes = BBoxTransform()
clipBoxes = ClipBoxes()

out = postprocess(x,
                  anchors, regression, classification,
                  regressBoxes, clipBoxes,
                  threshold, iou_threshold)

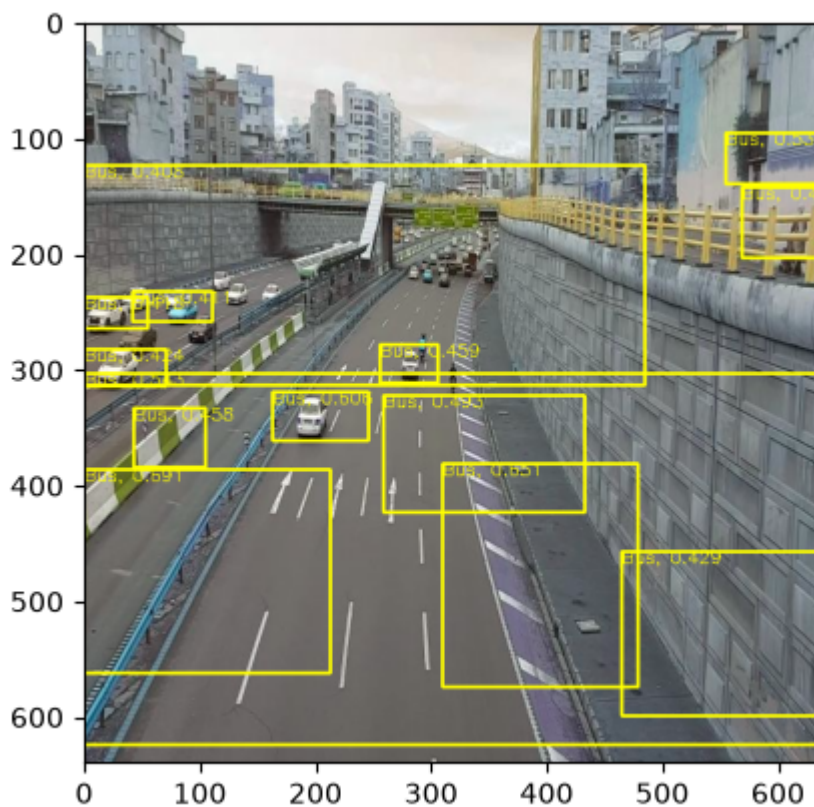
out = invert_affine(framed metas, out)

for i in range(len(ori_imgs)):
    if len(out[i]['rois']) == 0:
        continue
    ori_imgs[i] = ori_imgs[i].copy()
    for j in range(len(out[i]['rois'])):
        (x1, y1, x2, y2) = out[i]['rois'][j].astype(np.int32)
        cv2.rectangle(ori_imgs[i], (x1, y1), (x2, y2), (255, 255, 0), 2)
        obj = obj_list[out[i]['class_ids'][j]]
        score = float(out[i]['scores'][j])

        cv2.putText(ori_imgs[i], '{}', {:.3f}'.format(obj, score),
                    (x1, y1 + 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                    (255, 255, 0), 1)

    plt.imshow(ori_imgs[i])

```



In [60]: out

```
Out[60]: [{'rois': array([[ 0.          , 386.97754 , 213.03592 , 562.02386 ],
                        [310.61496 , 381.07074 , 479.41296 , 574.3063  ],
                        [163.30984 , 319.8896  , 246.9628  , 361.2873  ],
                        [ 0.          , 303.341   , 638.75    , 624.9462  ],
                        [555.6471  , 95.21561  , 638.75    , 139.08946 ],
                        [259.09216 , 322.94547 , 433.133   , 423.97186 ],
                        [256.66473 , 278.17706 , 306.92154 , 311.24753 ],
                        [ 43.372963, 333.03244 , 105.93053 , 384.1545  ],
                        [569.52734 , 142.94455 , 638.75    , 203.38693 ],
                        [465.3984  , 457.46094 , 638.75    , 599.78314 ],
                        [ 0.          , 282.6277  , 71.78121  , 314.86243 ],
                        [ 0.          , 237.50262 , 55.545414, 264.8605  ],
                        [ 42.52867 , 232.75436 , 111.66727 , 258.44916 ],
                        [ 0.          , 123.35731 , 485.82047 , 313.94058 ]], dtype=float32),
  'class_ids': array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int64),
  'scores': array([0.6908743 , 0.65087163, 0.60648894, 0.54299057, 0.5298389 ,
                  0.4932545 , 0.45855916, 0.45773023, 0.4422814 , 0.42888534,
                  0.42406186, 0.41760555, 0.410814  , 0.40786317], dtype=float32)}]
```

학습한 가중치로 test 이미지 한 장을 예측한다. Part 1과 같은 순서(전처리 → 모델 생성·로드 → 추론 → 후처리 → 좌표 복원 → 그리기)이고, 클래스 6개, `threshold = 0.4`, 앵커를 `ratios=` · `scales=` 로 직접 넘기는 점이 다르다. 원본 코드 그대로의 결과는 박스 14개가 모두 **Bus** 로 찍혔고, 건물·도로를 덮는 큰 박스가 많다. 하늘색이어야 할 박스는 노란색으로 보이고 사진 색도 뒤바뀌었다. 원인은 아래 코드 리뷰 3-4-3-10-3-11에서 확인한다.

🔍 코드 리뷰 3-10 · 학습과 다른 앵커로 예측

- **문제:** 앵커를 [(1.0, 1.0), (1.3, 0.8), (1.9, 0.5)] 와 기본 크기로 넘긴다. 학습(yml)은 [(0.52, 1.0), (0.67, 1.0), (0.82, 1.0)], [0.54, 1.0, 2.13] 이었다
- **영향:** 앵커는 가중치 파일에 저장되지 않아 **에러 없이 로드되지만**, 모델이 배운 "앵커를 얼마나 고칠지"가 다른 앵커에 적용되어 박스 위치·크기가 틀어진다
- **개선안:** yml 값(eval(params['anchors_ratios'])) 등을 그대로 쓴다 (아래 근거 2)

```
params = yaml.safe_load(open('projects/my_car_detect_proj.yml'))
model = EfficientDetBackbone(compound_coef=compound_coef, num_classes=len(obj_list),
                             ratios=eval(params['anchors_ratios']),
                             scales=eval(params['anchors_scales']))
```

🔍 코드 리뷰 3-11 · BGR 이미지를 RGB로 표시

- **문제:** ori_imgs 는 cv2.imread 로 읽은 BGR인데 plt.imshow 는 RGB로 해석한다
- **영향:** 하늘색 (255, 255, 0) 박스가 노란색으로 보이고 사진 색도 뒤바뀐다
- **개선안:** cv2.cvtColor(img, cv2.COLOR_BGR2RGB) 후 표시

```
plt.imshow(cv2.cvtColor(ori_imgs[i], cv2.COLOR_BGR2RGB))
```

🔍 코드 리뷰 3-12 · plt.imshow 가 반복문 안에 있음

- **문제:** plt.imshow 가 박스 반복문 안에 있다
- **영향:** 박스 수만큼 같은 그림을 다시 그린다
- **개선안:** 반복문 밖으로 뺀다

```
for i in range(len(ori_imgs)):
    for j in range(len(out[i]['rois'])):
        ... # 박스·글자 그리기
    plt.imshow(cv2.cvtColor(ori_imgs[i], cv2.COLOR_BGR2RGB)) # 박스를
    다 그린 뒤 한 번
plt.show()
```

🔍 리뷰 근거 확인

위 3-4-3-10-3-11이 실제 결과에 어떤 영향을 주는지 확인한다.

```
In [61]: # 🔍 리뷰 근거 1 (3-4): 학습 데이터의 카테고리명과 실제 박스에 쓰인 category_id
import json, collections
coco = json.load(open('datasets/my_car_detect_proj/annotations/instances_train.json', encoding='utf-8'))
print('categories:', [(c['id'], c['name']) for c in coco['categories']])
print('박스에 쓰인 category_id:', dict(sorted(collections.Counter(a['category_id'] for a in coco['annotations']).items())))
print('레포 라벨(category_id - 1) → 원본 obj_list 이름:', {cid: obj_list[cid - 1] for cid in range(1, 6)})
```

```
categories: [(0, 'cars'), (1, 'Bus'), (2, 'Car'), (3, 'Motorcycle'), (4, 'Pickup'), (5, 'Truck')]
```

박스에 쓰인 category_id: {1: 31, 2: 2023, 3: 355, 4: 251, 5: 64}

레포 라벨(category_id - 1) → 원본 obj_list 이름: {1: 'cars', 2: 'Bus', 3: 'Car', 4: 'Motorcycle', 5: 'Pickup'}

박스에 쓰인 번호는 1~5뿐이고 0번(cars)은 하나도 없다. 레포가 만드는 라벨(category_id - 1)로 원본 obj_list 를 찾으면, Bus(1)가 cars 로, Car(2)가 Bus 로 찍힌다. 데이터의 대부분인 Car(2023개)가 전부 Bus 라는 이름으로 표시된다는 뜻이다.

```

In [62]: # 리뷰 근거 2 (3-4.3-10.3-11): 정답 / 원본 예측 코드 / 앵커·이름·색을 고친 코드를 같은 사진으로 비교
import yaml
params = yaml.safe_load(open('projects/my_car_detect_proj.yml', encoding='utf-8'))

def predict(m, names, th=0.4):
    ori, framed, metas = preprocess(img_path, max_size=512)
    xx = torch.stack([torch.from_numpy(fi).cuda() for fi in framed], 0).to(torch.float32).permute(0, 3, 1, 2)
    with torch.no_grad():
        _, reg, cls, anc = m(xx)
        res = postprocess(xx, anc, reg, cls, BBoxTransform(), ClipBoxes(), th, iou_threshold)
        res = invert_affine(metas, res)[0]
        img = cv2.cvtColor(ori[0], cv2.COLOR_BGR2RGB)
        for (x1, y1, x2, y2), c, s in zip(np.array(res['rois']).astype(int).reshape(-1, 4), res['class_ids'], res['scores']):
            cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 255), 2)
            cv2.putText(img, f'{names[int(c)]} {s:.2f}', (x1, max(y1 - 3, 10)), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255), 1)
        return img, [names[int(c)] for c in res['class_ids']]

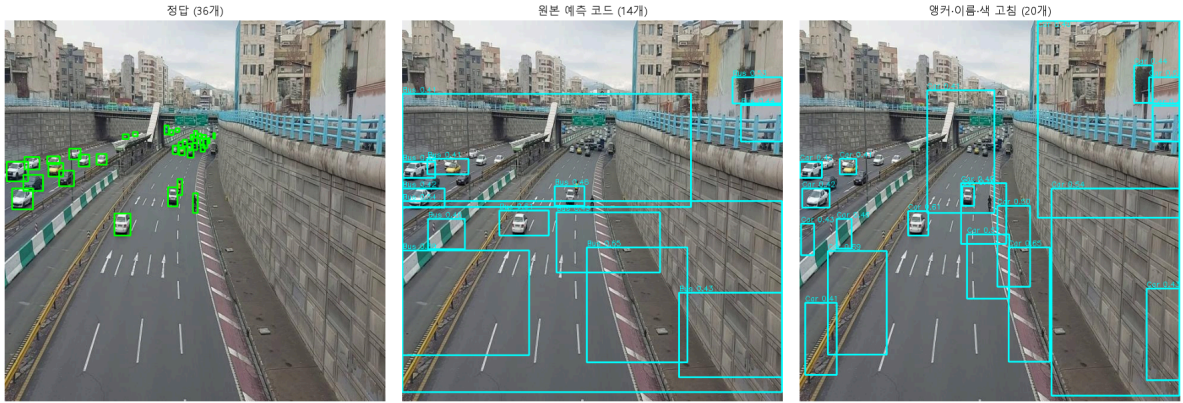
# 정답 박스 (같은 사진의 COCO 라벨)
gt_coco = json.load(open(os.path.join(os.path.dirname(img_path), '_annotations.coco.json'), encoding='utf-8'))
gt_img = [im for im in gt_coco['images'] if im['file_name'] == os.path.basename(img_path)][0]
gt_names = {c['id']: c['name'] for c in gt_coco['categories']}
gt = cv2.cvtColor(cv2.imread(img_path), cv2.COLOR_BGR2RGB)
gt_labels = []
for a in gt_coco['annotations']:
    if a['image_id'] == gt_img['id']:
        bx, by, bw, bh = map(int, a['bbox'])
        cv2.rectangle(gt, (bx, by), (bx + bw, by + bh), (0, 255, 0), 2)
        gt_labels.append(gt_names[a['category_id']])

# 원본 예측 코드: 위 셀의 model (예측 셀 앵커) + 원본 obj_list
before, before_labels = predict(model, obj_list)
# 고친 코드: yml(학습) 앵커 + 'cars'를 뺀 이름
model_fix = EfficientDetBackbone(compound_coef=0, num_classes=len(params['obj_list']),
                                ratios=eval(params['anchors_ratios']), scales=eval(params['anchors_scales']))
model_fix.load_state_dict(torch.load('logs/my_car_detect_proj/' + weight_file[-1]))
model_fix.requires_grad_(False).eval().cuda()
after, after_labels = predict(model_fix, params['obj_list'][1:])

plt.rcParams['font.family'] = 'Malgun Gothic' # 제목 한글이 깨지지 않게 그리기 전에 지정
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, im, title in zip(axes, [gt, before, after],
                        [f'정답 ({len(gt_labels)}개)', f'원본 예측 코드 ({len(before_labels)}개)', f'앵커·이름·색 고침 ({len(after_labels)}개)']):
    ax.imshow(im); ax.set_title(title); ax.axis('off')
plt.tight_layout(); plt.show()
print('정답      : ', dict(collections.Counter(gt_labels)))

```

```
print('원본 코드 :', dict(collections.Counter(before_labels)))
print('고친 코드 :', dict(collections.Counter(after_labels)))
```



정답 : {'Car': 28, 'Motorcycle': 4, 'Pickup': 4}
 원본 코드 : {'Bus': 14}
 고친 코드 : {'Car': 20}

같은 사진(정답 36개: Car 28, Motorcycle 4, Pickup 4)으로 비교했다.

- **원본 예측 코드:** 14개가 모두 **Bus** 다. 데이터 대부분인 Car가 이름 밀림(3-4)으로 **B**us로 찍혔고, 학습과 다른 앵커(3-10) 때문에 건물·도로를 덮는 큰 박스가 많다.
- **앵커·이름·색 고침:** 20개가 모두 **Car**로 이름이 맞고, 왼쪽 위 차들처럼 박스가 차에 맞게 그려진 경우가 생겼다. 색도 정상으로 보인다.
- 다만 고친 뒤에도 벽·건물에 큰 박스가 남고, 멀리 있는 작은 차는 못 찾는다.

→ 3-4-3-10-3-11을 고치면 이름과 박스 기준은 바로잡히지만, 모델 성능(AP 0.005)이 낮은 문제는 남는다. 이건 코드 실수가 아니라 작은 물체가 많은 데이터, 적은 학습 데이터(136장), 흑백으로 학습하고 컬러로 예측한 것(3-1) 때문으로 본다.

코드 리뷰 종합

중요도	항목	한 줄 요약
● 높음	3-10 앵커 불일치	학습과 다른 앵커로 예측해 박스가 틀어진다. 에러가 나지 않아 알아차리기 어렵다
● 높음	3-4 클래스 이름 밀림	<code>obj_list</code> 에 상위 카테고리 <code>cars</code> 가 들어가 이름이 한 칸씩 밀린다
● 높음	3-1 데이터 버전 혼용	분석은 컬러, 학습은 흑백, 예측은 다시 컬러로 한다
● 중간	3-3 yml 메모 불일치	메모대로 만들면 데이터를 못 찾고 CPU로 학습한다
● 중간	3-5-3-6-3-7 2단계 학습 설정	파일 이름 하드코딩, 실제 20에폭, 같은 학습률
● 중간	3-9 valid로만 평가	test 셋으로 채점하지 않는다
● 중간	2-1 영상 확인 결과 무시	파일이 없어도 그대로 진행한다
● 낮음	1-1-1-2 설정값 미반영	<code>compound_coef</code> , 앵커 변수를 정의만 하고 모델에 넘기지 않는다
● 낮음	2-3-2-4-3-11-3-12 그리기 코드	반복문 안 <code>return</code> , <code>display</code> 두 번 호출, BGR 표시, <code>imshow</code> 위치
● 낮음	1-3-1-6-2-2-2-7-3-2 중복·비효율	추론 두 번, 같은 코드 세 번, 모델 재정의, 프레임 저장 후 다시 합치기, 폴더 변환 두 번

공통 원인은 **같은 값을 여러 곳에 손으로 적은 것이다**. 앵커·클래스 목록·데이터 경로·가중치 파일 이름을 각 셀에 따로 적다 보니, 한 곳이 바뀌어도 다른 곳은 그대로 남았다. yml 하나를 기준으로 삼고 나머지 셀이 그 값을 읽어 쓰게 하면 3-1-3-3-3-4-3-5-3-10이 한꺼번에 해결된다.

목표 기준 평가

목표	수행 내용	결과
① 추론 과정 이해	전처리(<code>framed metas</code>) → 모델 출력 4가지(49104개 앵커) → 후처리(NMS) → 좌표 복원을 단계별로 확인	37개 박스로 거리 사진의 차·사람 탐지
② 이미지·영상 분석	사전학습 D0로 이미지 1장, <code>soccer.gif</code> 367프레임 분석 후 mp4로 합치기	사람·벤치는 잘 잡았지만 축구공은 못 잡음, 367프레임을 mp4로 합침
③ 학습·평가	흑백 차량 데이터(train 136장)로 head 10에폭 → 전체 20에폭 학습, COCO 방식 평가	검증 AP50 0.011 (AP 0.005). 학습은 진행됐지만 작은 차를 거의 찾지 못함

목표	수행 내용	결과
④ 코드 리뷰	문제가 있는 코드 바로 아래에 리뷰 25개 항목, 주요 문제는 실행 결과로 근거 확인	앵커 불일치·클래스 이름 밀림·데이터 버전 혼용이 가장 큰 문제

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
원본 노트북을 이 PC에서 그대로 실행할 수 없음	원본은 Colab용이라 /content/... 경 로, wget , mkdir - p 가 들어 있다	이 PC에서 안 되 는 곳만 고치고 코랩용 과 노트북용 을 나란히 표시 했다. 원본 코드 셀 57개 중 43개 는 글자 하나까 지 그대로다
!ls -l , !pwd 가 'ls' is not recognize d... 에러	이 PC의 Jupyter는 ! 명령을 Windows 명령창(cmd)으로 실행해서 ls · pwd · un zip · grep 이 없다. 처음엔 Git Bash에서 시험해 된다고 잘못 판단했다	os.getcwd() , os.listdir , zipfile , dir /b /od 로 바꿨 다
soccer.mp4 가 없음	원본 Colab 실행 기록 에도 False 였다. 실 제로 쓰인 파일은 so ccer.gif 였다	soccer.gif 를 받아 video_sr c 를 바꿨다
yaml 메모대로 하면 데이터를 못 찾음	project_name 이 폴 더 이름과 달랐다	my_car_detect _proj 로 맞췄 다. 원본 Colab 로그도 이 이름 의 폴더에 저장 되어 있었다
num_gpus: 0 이면 CPU로 학습함	train.py 가 0이면 GPU를 끈다	num_gpus: 1 로 바꿨다
학습이 시작되지 않음 (UnicodeDecodeError: 'cp949')	yaml 파일 안에 한글· 이모지 주석을 넣었더 니 train.py 가 yaml 을 cp949로 읽다가 실패했다. ! 명령은 실패해도 셀이 멈추지 않아 다음 셀에서야 드러났다	yaml은 파이썬으 로 쓰고 파일 안 에는 원본 영어 주석만 남겼다
ReduceLROnPlateau(..., verbose=True) 에러	최신 PyTorch에서 ve rbose 인자가 없어 졌다	원본 안내대로 지웠다

막힌 부분	원인	해결
학습이 에폭당 3분 30초 걸림	워커 12개를 Windows에서는 에폭마다 새로 띄운다. 그 동안 GPU 사용률은 0%였다	-n 0 으로 에폭당 약 15초가 됐다
! python train.py 실패	이 PC에서 python 은 pycocotools 가 없는 시스템 파이썬을 가리킨다	!{sys.executable} 로 커널의 파이썬을 썼다
PCB 사진 추론에서 'NoneType' object is not subscriptable	커널이 이미 레포 안에 있는 상태에서 git clone · os.chdir 를 다시 실행해 레포가 두 겹으로 받아졌고, 안쪽 사본에는 이미지가 없었다	안쪽 사본을 지우고 커널을 재시작해 처음부터 다시 실행했다. 흐름 1은 커널 재시작 직후에만 실행한다

개인 회고

실습을 통해 알게 된 것

이번 실습에서 가장 크게 남은 것은 탐지 모델 안에서 실제로 어떤 일이 일어나는지에 대한 이론이다. 먼저 특징맵 P3~P7의 숫자 N은 이미지를 2^N 배로 줄여서 본다는 뜻이고, 많이 줄인 P7은 칸 하나가 원본의 넓은 영역을 요약해서 보기 때문에 큰 물체를, 칸이 촘촘한 P3은 작은 물체를 맡는다는 것을 알았다. P3은 위치가 정확하지만 의미 정보가 약하고 P7은 그 반대라서, EfficientDet은 BiFPN으로 둘을 섞어 서로의 약점을 보완한다.

앵커의 역할도 이해했다. 앵커는 특징맵의 칸마다 미리 깔아 두는 기준 박스로, 모델은 박스를 처음부터 그리지 않고 앵커를 기준으로 얼마나 옮기고 늘릴지만 예측한다. 앵커는 학습과 예측 내내 바뀌지 않고, 칸마다 9개씩 깔려서 D0에서는 49,104개가 된다. 학습할 때는 앵커와 정답 박스의 IoU로 역할을 정하는데, 0.5 이상이면 그 물체를 배우고 0.4 미만이면 배경으로 배우며, 그 사이의 애매한 앵커는 학습에서 뺀다. 모델이 정답을 스스로 아는 것이 아니라, 사람이 만든 라벨을 손실 함수가 예측과 비교해 얼마나 틀렸는지 알려주는 구조라는 것도 이때 알게 됐다.

마지막으로 Focal Loss가 왜 필요한지 알게 됐다. 찾으려는 물체에 비해 배경 앵커가 압도적으로 많아서, 그냥 학습하면 배경의 손실이 쌓여 물체의 손실이 묻혀 버린다. Focal Loss는 이미 잘 맞힌 쉬운 배경의 손실을 크게 줄여서, 모델이 어려운 물체를 배우는 데 집중하게 한다.

어려웠던 것

특징맵에서 왜 P7이 큰 물체를, P3이 작은 물체를 맡는지 이해하기 어려웠다. 처음에는 이미지를 줄여서 보는데 왜 큰 물체를 담당하는지 납득이 되지 않았는데, 칸 하나가 원본의 얼마만큼을 요약해서 보는지로 생각하니 이해가 됐다. Colab용 원본 코드를 내 컴퓨터에서 돌리는 것도 어려웠다. 경로, 리눅스 명령, 파이썬 위치처럼 환경 차이 때문에 막히는 곳이 많았다.

좋았던 점과 아쉬웠던 점

이론적인 부분을 많이 알아간 것 같아서 기분이 좋았다. 반면 Colab 코드를 내 컴퓨터에 맞게 바꾸는 데 시간이 많이 들었고, 이론 내용을 한 번에 이해하지 못했다. 듣고 이해한 것 같아도 막상 배운 내용을 내 말로 설명하려고 하면 막혔다.

처음 기대와 실제

솔직히 처음에는 큰 기대가 없었다. 그런데 막상 해 보니 이론적인 내용이 흥미로워서 공부하는 것이 즐거웠다. 교수님 말씀대로 중학생도 이해할 수 있게 설명하는 훈련을 해야겠다고 느꼈고, 이번 실습에서 얻은 것도 새로운 지식과 함께 바로 이 마음가짐이다.

현장에 쓴다면

솔직히 지금은 YOLO와 크게 다르지 않아 보이고, 개인적으로는 YOLO가 더 좋아 보여서 현장에서는 YOLO를 쓸 것 같다. 다만 자원이 부족한 곳에서는 적은 연산량으로 높은 효율을 내는 EfficientDet이 더 좋을 것 같다.

앞으로 해 볼 것

앞으로 있을 중간고사에 대비해 배운 내용을 쉽게 설명하는 방법을 연습할 예정이다. 또 입력 이미지 크기를 바꿔(D1은 640, D2는 768) 학습해 보면서 앵커 개수가 어떻게 달라지는지 직접 구해 보고 싶다.

동료 피드백

받은 피드백은 없다.